
WalletWatch

Dossier de projet

Geoffrey Carpentier

Titre professionnel **Développeur Web et Web Mobile**

La Plateforme, session de septembre 2026

Application de suivi de patrimoine

Version révisée le 11 septembre 2026. Code de référence : a8a5e6d

Sommaire

• Introduction	3
• Partie 1 : Expression des besoins : objectifs et limites du projet	4
◦ 1.1 Contexte et problématique	4
◦ 1.2 Objectifs, public visé et périmètre	4
◦ 1.3 Cas d'utilisation et parcours	6
◦ 1.4 Contraintes, méthode de travail et organisation	8
• Partie 2 : Environnement technique	10
◦ 2.1 Choix de la pile technique	10
◦ 2.2 Architecture générale	11
◦ 2.3 Environnement de développement et outillage	13
• Partie 3 : Conception et réalisation du projet	15
◦ 3.1 Maquettage des interfaces	15
◦ 3.2 Réalisation des interfaces statiques	18
◦ 3.3 Réalisation des interfaces dynamiques	19
◦ 3.4 Mise en place de la base de données relationnelle	24
◦ 3.5 Composants d'accès aux données	28
◦ 3.6 Composants métier côté serveur	31
◦ 3.7 Éléments de sécurité de l'application	35
◦ 3.8 Jeu d'essai de la fonctionnalité la plus représentative	36
◦ 3.9 Tests et qualité du code	38
◦ 3.10 Veille sur les vulnérabilités de sécurité	40
◦ 3.11 Déploiement de l'application	41
• Partie 4 : Bilan du projet	44
◦ 4.1 Difficultés rencontrées et solutions apportées	44
◦ 4.2 Bilan et perspectives	45
• Annexes	46
◦ Annexe A : Maquettes des interfaces	46
◦ Annexe B : Structure de la base	51
◦ Annexe C : Points d'entrée de l'API	51
◦ Annexe D : Extraits techniques complémentaires	52
◦ Annexe E : Captures complémentaires	53
◦ Annexe F : Scénarios du jeu d'essai	61

Introduction

WalletWatch est l'application de suivi de patrimoine que j'ai réalisée pendant ma formation au titre professionnel Développeur Web et Web Mobile. Elle regroupe sur un même tableau de bord des cryptomonnaies, des devises, des métaux précieux et une sélection d'actions. À partir des mouvements saisis, elle récupère les cours et calcule la quantité détenue, le **prix de revient moyen pondéré** et les plus-values. L'interface s'adapte au téléphone et au bureau.

J'ai réalisé ce projet individuellement, sans client ni commanditaire, de juillet à septembre 2026. J'ai rédigé l'expression des besoins et défini un **périmètre réalisable** dans le temps disponible, depuis les maquettes et la base de données jusqu'aux tests et à la documentation du déploiement.

Le code examiné est celui de la branche dev, au [commit a8a5e6d sur GitHub](#). Les résultats de tests sont datés dans la section 3.9. Les captures ont été refaites le 10 septembre 2026 sur l'instance locale de démonstration, avec le compte fictif Camille Durand. Les maquettes et les traces de développement sont présentées comme des documents historiques. Les cours peuvent varier entre deux consultations ; les nombres affichés ne sont pas des cours de référence.

Le produit a pris le nom WalletWatch en fin de projet. Certains identifiants techniques et les maquettes historiques conservent le nom CapitAll. J'ai conservé ces *traces de conception* sans les présenter comme des captures récentes de l'application.

Partie 1 : Expression des besoins : objectifs et limites du projet

1.1 Contexte et problématique

Un suivi réparti entre plusieurs outils

Le besoin de départ est de réunir les informations d'un patrimoine réparti entre plusieurs supports. Un particulier peut consulter ses cryptomonnaies sur une plateforme, ses devises sur une autre et ses actions chez un courtier. Il doit ensuite rapprocher ces informations pour obtenir une vue d'ensemble.

Un tableur permet ce regroupement, mais il faut préparer les formules, actualiser les cours et vérifier les saisies. Ce travail peut devenir contraignant dès que plusieurs achats et ventes concernent le même actif. WalletWatch cherche à automatiser ce suivi pour les actifs pris en charge ; le projet ne prétend pas remplacer tous les outils existants.

Les calculs à rendre compréhensibles

Le prix de revient n'est pas le prix du dernier achat. Dans le modèle retenu, il correspond au coût des achats, frais compris, rapporté à la quantité acquise. Par exemple, dix titres achetés à 100 € avec 5 € de frais, puis dix titres à 120 € avec 5 € de frais donnent un prix de revient de 110,50 € par titre.

Une vente réduit la quantité et produit un résultat réalisé, sans modifier le prix de revient des titres restants. Si l'utilisateur ajoute ou corrige un mouvement ancien, il faut reprendre l'historique dans l'ordre des dates. Ces règles ont guidé le moteur de calcul et les jeux d'essai présentés en 3.6 et 3.8.

Les cours ne changent pas tous au même rythme. L'application doit donc distinguer un cours disponible, un dernier cours connu et l'absence de cours. Présenter une position sans cours comme valant zéro donnerait une information trompeuse.

Le besoin formulé

Permettre à un particulier de consulter la valeur des actifs qu'il suit, leur répartition et ses plus-values, sans recopier les cours à chaque consultation. La saisie doit aussi refuser les opérations incohérentes, notamment une vente qui dépasse la quantité disponible à sa date.

Cette valeur reste celle des actifs enregistrés dans l'application. WalletWatch ne suit pas les comptes bancaires ni le solde en euros provenant d'une vente. Il ne fournit pas de conseil d'investissement ni de calcul fiscal.

Un projet de formation

J'ai choisi ce sujet le 9 juillet 2026, car il permet de travailler les interfaces, les données, la sécurité et les calculs côté serveur autour d'un même besoin. J'ai fixé les limites du projet avant de développer les parcours. Les services de cours disposent d'offres gratuites, mais leur disponibilité et leurs quotas restent des contraintes extérieures.

Le cadrage a commencé le 9 juillet et le développement le 24 juillet, avec une première préparation des livrables le 7 septembre, puis une révision du dossier le 10 septembre. La section 1.4 présente l'organisation et les ajustements du calendrier.

1.2 Objectifs, public visé et périmètre

Public et objectifs

L'application s'adresse à un particulier qui souhaite regrouper le suivi de ses actifs. J'ai retenu un usage fréquent sur téléphone et un volume modeste : quelques dizaines de positions et quelques centaines de mouvements par compte. Il s'agit d'*hypothèses de conception*, pas des conclusions d'une étude auprès

d'utilisateurs.

Les objectifs sont de centraliser les quatre classes d'actifs, récupérer leurs cours, calculer un prix de revient tenant compte des frais et présenter les résultats de manière lisible. Les données de chaque compte doivent rester accessibles uniquement à son propriétaire.

Fonctions livrées

Besoin	Réponse de l'application
Accéder à son suivi	Inscription, connexion, session protégée, changement de mot de passe et récupération limitée au contexte de démonstration
Suivre ses actifs	Création d'une position, renommage, liste filtrable et détail avec historique
Tenir les mouvements à jour	Achat, vente, sortie sans vente, correction et suppression avec contrôles serveur
Comprendre les frais	Frais en euros, dans l'actif ou dans un autre actif avec contre-valeur en euros
Consulter son patrimoine	Valeur des positions, prix de revient, plus-values, répartition et relevés historiques
Suivre un niveau de cours	Seuil sur un actif ou sur le patrimoine, évalué lors d'une actualisation
Lire et récupérer ses données	Affichage en euros ou dollars, masquage des montants, export CSV des mouvements

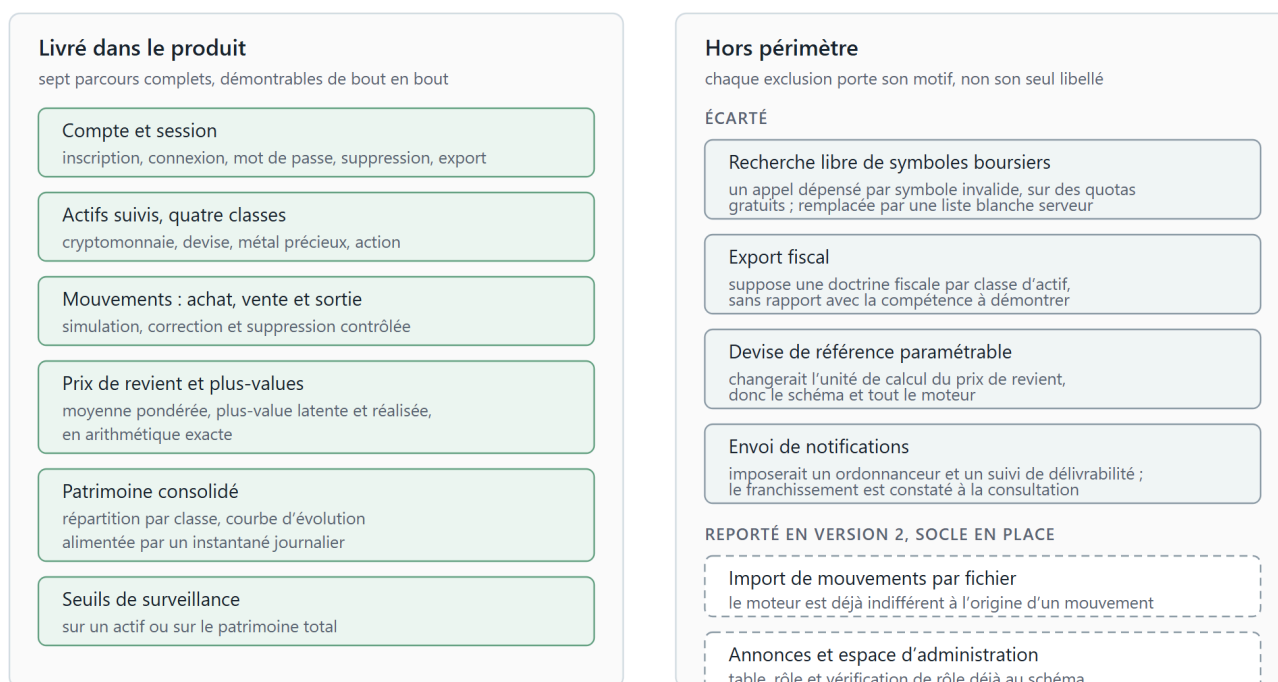


Figure 1 : Le périmètre livré et ses exclusions. Chaque exclusion porte son motif ; le trait pointillé distingue ce qui est reporté en version 2, socle déjà en place, de ce qui est écarté.

Limites

Le catalogue d'actions est limité à 106 symboles autorisés dans le code. La liste et les fournisseurs de secours ont été choisis pendant le projet en fonction des accès disponibles. Leur présence dans le catalogue ne garantit pas qu'un cours soit disponible à chaque consultation. Les métaux proposés sont l'or, l'argent, le platine et le palladium. Les suggestions de cryptomonnaies et de devises n'interdisent pas de saisir un autre symbole pris en charge par le fournisseur.

WalletWatch ne passe aucun ordre et ne suit pas les liquidités. Les euros reçus lors d'une vente ne deviennent donc pas un solde dans l'application. Une baisse du total peut venir d'une vente ou d'une sortie, et pas seulement d'une baisse des cours. La courbe présente l'évolution de la valeur des actifs suivis ; ce n'est pas une mesure de rendement corrigée des apports et retraits.

Les relevés sont enregistrés lors de l'actualisation, au plus une fois par jour. Si l'utilisateur ne consulte pas l'application, les jours manquants ne sont pas reconstitués. Un cours indisponible peut être remplacé par le *dernier cours connu*, clairement daté. S'il n'existe aucune valeur de secours, la position reste non valorisée et son absence du total est signalée.

Les seuils ne sont pas surveillés en continu et n'envoient pas de courriel. Les annonces et l'administration sont reportées, même si le code contient des éléments préparatoires. L'import de mouvements, les documents fiscaux et les conseils d'investissement sont hors du périmètre.

Livrables

Le projet comprend le code de l'interface et de l'API, les scripts de base de données, les tests, les fichiers Docker et la documentation de déploiement. Ce dossier présente les réalisations significatives ; le support de maintenance est un livrable distinct.

1.3 Cas d'utilisation et parcours

Acteurs

J'ai identifié trois acteurs : le visiteur, l'utilisateur inscrit et les fournisseurs de cours.

Le *visiteur* n'est pas authentifié. Il peut créer un compte ou ouvrir une session, mais n'accède à aucune donnée patrimoniale : il n'existe ni portefeuille de démonstration ni page publique.

L'*utilisateur inscrit* est l'acteur principal. Il agit uniquement sur ses propres données, jamais sur celles d'un autre compte (voir 3.5).

Les *fournisseurs de cours* sont les acteurs externes : Coinbase pour les cryptomonnaies, Frankfurter pour les devises, gold-api.com pour les métaux, puis FMP, Finnhub et Alpha Vantage pour les actions. Seul le serveur les interroge ; le navigateur ne les appelle jamais directement (voir 2.2).

Un acteur d'administration a été écarté. Le schéma porte bien un rôle sur le compte, mais aucun cas d'utilisation ne lui est associé dans le périmètre livré (voir 1.2).

Besoins exprimés en user stories

En tant que...	Je veux...	Afin de...
visiteur	créer un compte ou me connecter	accéder à mon espace personnel
utilisateur inscrit	ajouter, modifier ou retirer un actif	organiser les positions que je souhaite suivre
utilisateur inscrit	enregistrer un achat, une vente ou une sortie	obtenir une quantité et un prix de revient à jour
utilisateur inscrit	consulter mon patrimoine et le détail d'une position	comprendre sa valeur et son évolution
utilisateur inscrit	créer et consulter des seuils	repérer les valeurs qui ont franchi un niveau choisi
utilisateur inscrit	modifier mon mot de passe, exporter mes mouvements ou supprimer mon compte	gérer mes données personnelles

Ces actions exigent une session ouverte, sauf l'inscription et la connexion. Les requêtes vérifient que l'actif ou le seuil appartient bien à l'utilisateur. Une vente est refusée si la quantité disponible est insuffisante à sa date, et la suppression du compte demande de saisir de nouveau le mot de passe. Une ressource appartenant à un autre compte est traitée comme inexistante.

L'écran de compte a été ajouté après la première version du modèle. Le vocabulaire a aussi évolué : la spécification fonctionnelle a remplacé *transaction* et *alerte* par *mouvement* et *seuil*. Le dossier utilise les termes visibles dans l'interface, tandis que la base et les adresses de l'API gardent les noms techniques d'origine.

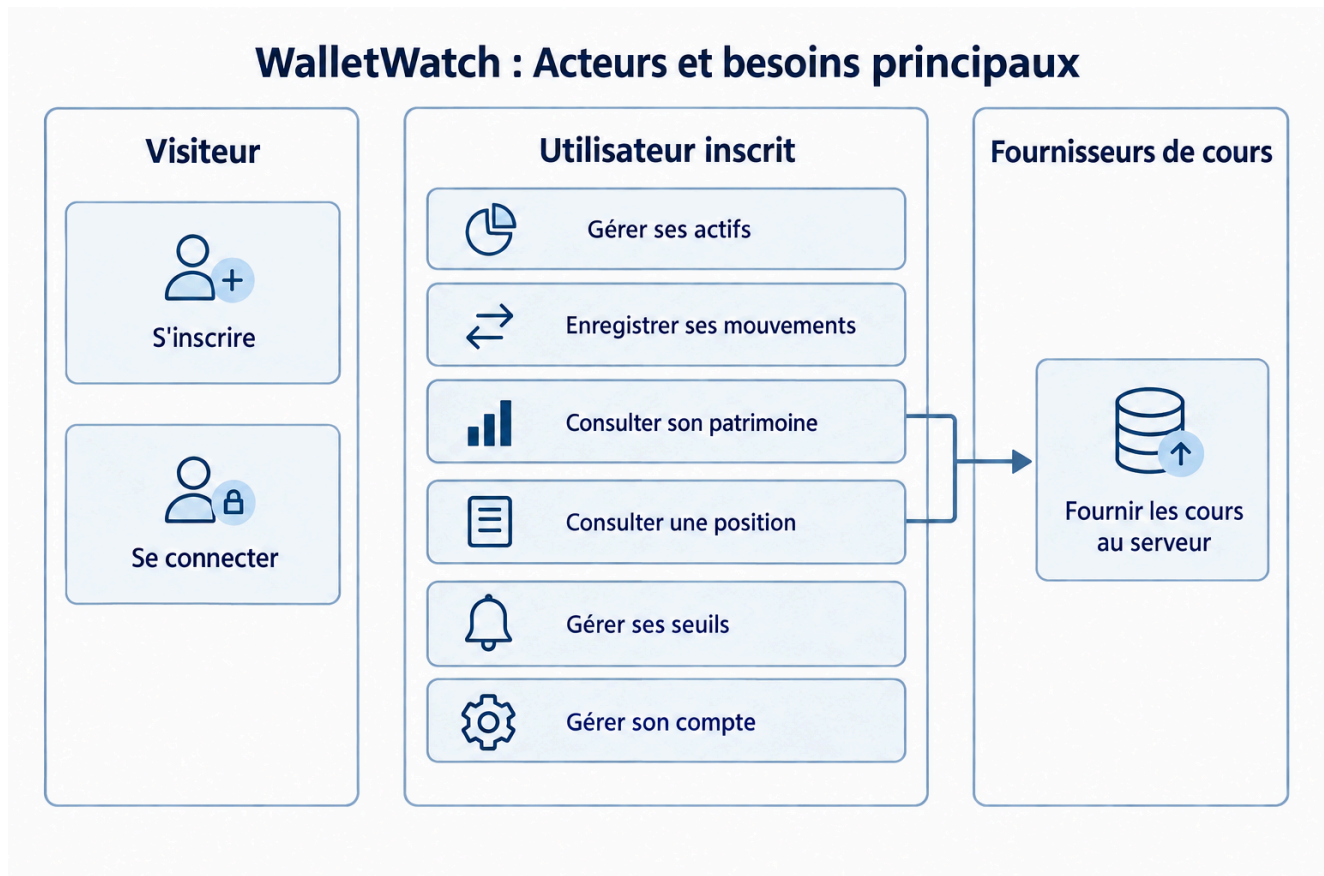


Figure 2 : Acteurs et besoins principaux. Les cours sont demandés par le serveur lors de la consultation du patrimoine ou d'une position.

Le diagramme regroupe les besoins par acteur. Les fournisseurs n'échangent qu'avec le serveur, qui demande un cours lorsqu'il doit valoriser le patrimoine ou une position.

J'ai regroupé cette récupération, car le tableau de bord et le détail d'une position utilisent le même cours, le même cache et la même valeur de secours lorsqu'un fournisseur ne répond pas. Les deux parcours partagent ainsi une seule implémentation (voir 3.6).

La consultation du tableau de bord déclenche aussi deux traitements internes : elle évalue les seuils actifs et enregistre la valorisation du jour si elle n'existe pas encore. Ce second traitement alimente la courbe d'évolution sans tâche planifiée supplémentaire.

Parcours

WalletWatch : Enchaînement des écrans

Sept écrans, aucune impasse. Les quatre parcours principaux sont détaillés en bas de planche.

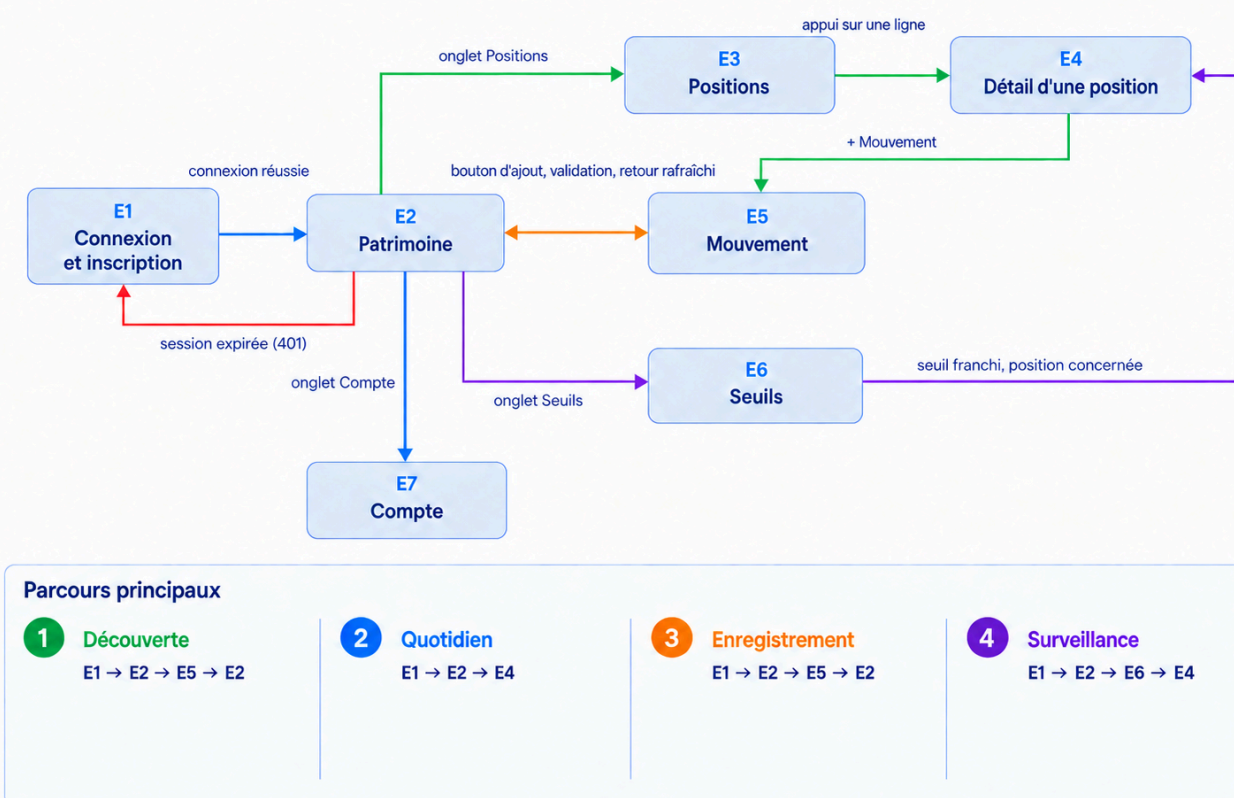


Figure 3 : Enchaînement des sept écrans et repérage des parcours Découverte, Quotidien, Enregistrement et Surveillance.

Le schéma relie les sept écrans. Chaque parcours permet de revenir au tableau de bord, tandis que la saisie s'ouvre au-dessus de l'écran courant. Quatre parcours y sont représentés.

La **découverte** va de l'inscription à un tableau de bord renseigné, en passant par l'état de premier lancement qui remplace la répartition vide par une action unique (voir 3.1).

Le **quotidien** est le parcours le plus fréquent et le plus court : connexion, tableau de bord, éventuellement le détail d'une position. Aucune saisie. C'est lui qui a dicté la hiérarchie de l'écran d'accueil, où le patrimoine total domine tout le reste.

L'**enregistrement** peut partir de n'importe quel écran avec le bouton d'ajout. Selon la largeur disponible, il ouvre une feuille glissante ou une boîte de dialogue, puis affiche un récapitulatif calculé par le serveur. Après validation, l'écran d'origine est rafraîchi. J'ai choisi ce composant superposé pour garder le contexte visible pendant la saisie.

La **surveillance** enchaîne le tableau de bord, les seuils franchis mis en évidence, et le détail de la position concernée. Elle n'implique aucun envoi : un franchissement se constate à la consultation.

1.4 Contraintes, méthode de travail et organisation

J'ai réalisé ce projet individuellement pendant la formation. Le cadrage a commencé en juillet 2026, puis le développement s'est réparti entre la base, l'API, les maquettes, l'interface et les vérifications. Le temps disponible m'a conduit à limiter le périmètre, notamment sur l'administration, les notifications et l'import de données.

J'ai organisé le travail en lots fonctionnels. Pour l'authentification, par exemple, l'inscription, la connexion et le contrôle du jeton devaient fonctionner ensemble avant de passer aux parcours privés. Les règles métier et les premiers traitements serveur ont précédé l'intégration des écrans. Cette progression m'a

permis de comparer les données attendues par l'interface avec les réponses réellement fournies par l'API.

Le suivi utilise des tickets, des branches et des demandes de fusion. Les commits indiquent le type de changement, la partie concernée et, lorsque c'est utile, le numéro du ticket. Par exemple : feat(back) : ajoute la connexion PostgreSQL par pool (#56). J'ai aussi noté les raisons des choix qui touchent plusieurs parties du projet, comme le calcul côté serveur, l'ordre des mouvements ou la réponse renvoyée pour une ressource appartenant à un autre compte.

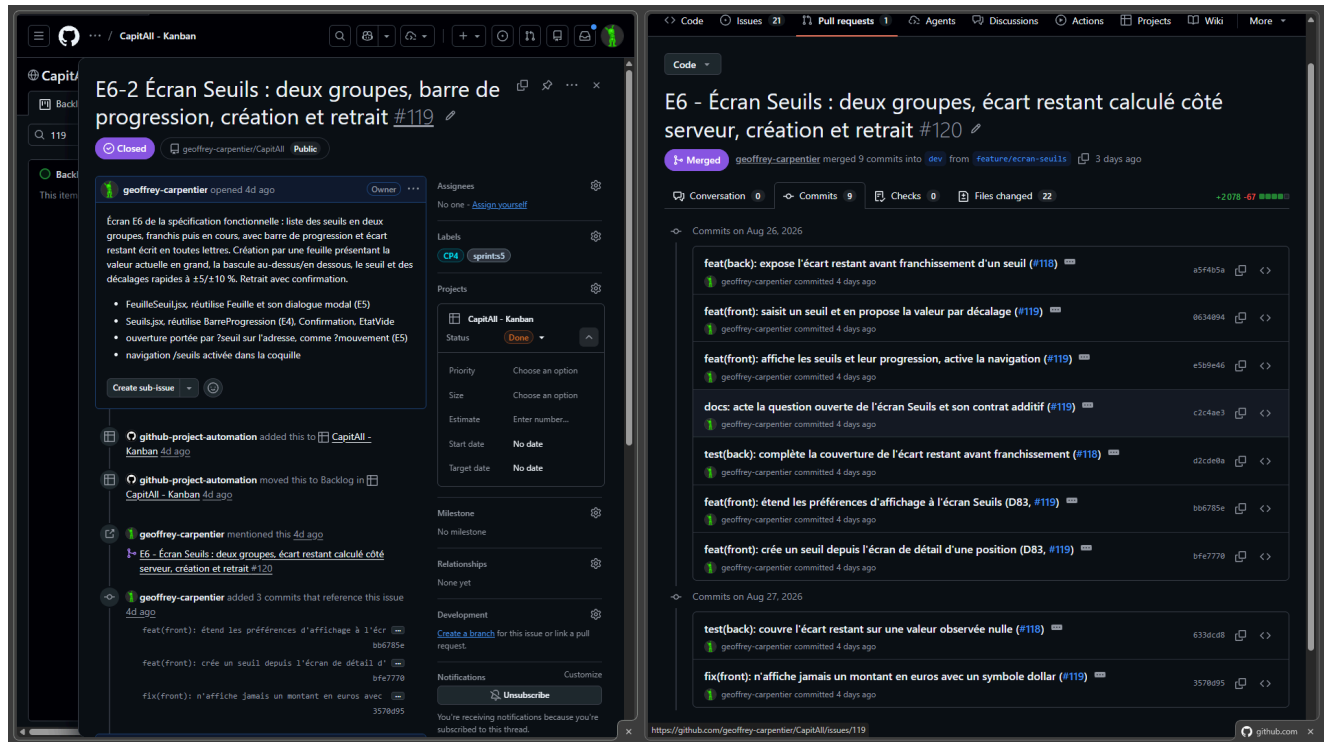


Figure 4 : Traçabilité d'un lot : l'issue à gauche, ses commits et la pull request qui la ferme à droite. Chaque message de commit porte le numéro de l'issue.

Cette figure est une trace du développement, pas l'état du tableau de suivi au jour de remise. Les corrections de fin de projet ont utilisé la branche rev ; ce dossier se réfère à la branche locale dev, au commit indiqué dans l'introduction.

Les fournisseurs gratuits imposent des quotas et peuvent être indisponibles. J'ai donc prévu un cache et un dernier cours connu. L'interface indique clairement quand ce cours n'est pas récent. Les tests du moteur, les essais avec PostgreSQL et les vérifications dans le navigateur ne contrôlent pas la même chose ; je les ai utilisés ensemble.

Partie 2 : Environnement technique

2.1 Choix de la pile technique

JavaScript côté client et côté serveur

J'ai choisi JavaScript pour l'interface et pour le serveur afin de travailler avec un langage déjà pratiqué en formation. Cela m'a permis de garder des habitudes communes pour les objets, les modules et les tests. PHP aurait aussi été possible, mais j'ai retenu cette pile pour ce projet.

JavaScript est un langage à typage dynamique. Les données reçues par HTTP restent des entrées non fiables : le serveur les valide avec Zod avant de les utiliser. Ajouter un typage statique au code ne dispenserait pas de cette validation à l'exécution.

Express pour organiser l'API

Express gère les routes de l'API et les contrôles exécutés avant chaque traitement. J'ai séparé le serveur en routes, contrôleurs, services et modèles. Je peux ainsi distinguer les échanges HTTP, les règles de calcul et les requêtes SQL, puis retrouver plus facilement l'origine d'une erreur. Ce découpage est détaillé en 2.2.

React et Vite pour l'interface

React gère les composants et l'état des écrans : période affichée, filtres, formulaire ouvert ou message d'erreur. Les interactions peuvent ainsi actualiser une partie de l'interface sans recharger toute la page. Une application rendue côté serveur pourrait également employer JavaScript pour obtenir ce comportement.

Vite lance le serveur de développement et construit les fichiers de l'interface. En développement, il relaie les appels `/api` vers Express. Dans la pile conteneurisée, Nginx sert les fichiers construits et relaie ces mêmes appels. Le navigateur utilise une seule origine.

PostgreSQL pour les données relationnelles

Le projet relie des comptes, des actifs, des mouvements et des seuils. PostgreSQL fournit les clés étrangères, les contraintes et les transactions nécessaires à ce modèle. Les colonnes `NUMERIC` permettent de stocker des valeurs décimales exactes dans les limites de précision définies par le schéma.

Le pilote pg restitue ces colonnes sous forme de chaînes. Le moteur les transforme en entiers `BigInt` avec une échelle explicite. Cette chaîne évite les approximations binaires d'un `Number` pour les calculs métier. Les divisions et les changements d'échelle suivent une règle d'arrondi documentée.

MySQL propose également un type décimal exact, `DECIMAL`. Le choix de PostgreSQL ne repose donc pas sur une absence de cette capacité chez MySQL. J'ai aussi utilisé ses index partiels et son type `TIMESTAMPZ`. Ce dernier représente un instant et l'affiche selon le fuseau de la session ; il ne conserve pas le nom du fuseau d'origine.

Difficulté rencontrée. Le port PostgreSQL habituel était déjà utilisé sur mon poste. Le fichier `Compose` permet de choisir un port dédié avec `POSTGRES_PORT`.

Redis pour limiter les appels de cours

Sans cache, consulter plusieurs positions entraînerait autant d'appels aux fournisseurs. Redis conserve temporairement les cours et garde aussi le dernier cours connu comme valeur de secours. Ce cache est partagé entre les utilisateurs de l'API. PostgreSQL reste chargé des données permanentes de l'application.

Une mémoire interne à Node aurait été plus simple au départ, mais son contenu disparaîtrait au redémarrage et ne serait pas partagé entre plusieurs processus. Si Redis est indisponible, le service tente directement les fournisseurs. Les limites de ce fonctionnement sont expliquées en 3.5.

Validation et tests

Zod décrit les entrées attendues et rejette les champs non autorisés. Les quantités et les prix restent des chaînes décimales. Vitest exécute les tests du serveur et de l'interface. Docker Compose fournit les services nécessaires avec des versions définies dans les fichiers de composition.

Versions de référence

Les versions ci-dessous proviennent des fichiers de verrouillage du dépôt examiné le 10 septembre 2026. Elles ne doivent pas être confondues avec les plages de versions déclarées dans `package.json`.

Élément	Version de référence	Source
Express	4.22.2	backend/package-lock.json
Pilote PostgreSQL pg	8.22.0	backend/package-lock.json
React	18.3.1	frontend/package-lock.json
React Router DOM	7.18.2	frontend/package-lock.json
Recharts	3.10.1	frontend/package-lock.json
Vite	8.1.5	frontend/package-lock.json
Vitest	4.1.10	Les deux fichiers de verrouillage
Oxlint serveur	1.71.0	backend/package-lock.json
Oxlint interface	1.75.0	frontend/package-lock.json
Node.js des images de construction et du serveur	Branche 22 Alpine	Dockerfiles
PostgreSQL	Branche 16 Alpine	Docker Compose
Redis	Branche 7 Alpine	Docker Compose
Nginx	Branche 1.27 Alpine	Dockerfile de l'interface

Les étiquettes d'images Docker indiquent les branches retenues ; sans digest figé, elles ne désignent pas à elles seules un contenu immuable. Le contrôle local du 10 septembre utilise Node.js 24.13.0 ; cette version du poste ne remplace pas celle choisie pour les images.

2.2 Architecture générale

L'application comporte une interface React et une API Express. Le navigateur appelle l'API ; il ne contacte ni PostgreSQL, ni Redis, ni les fournisseurs de cours. Ce choix garde les clés des fournisseurs côté serveur et permet de partager le cache entre les utilisateurs.

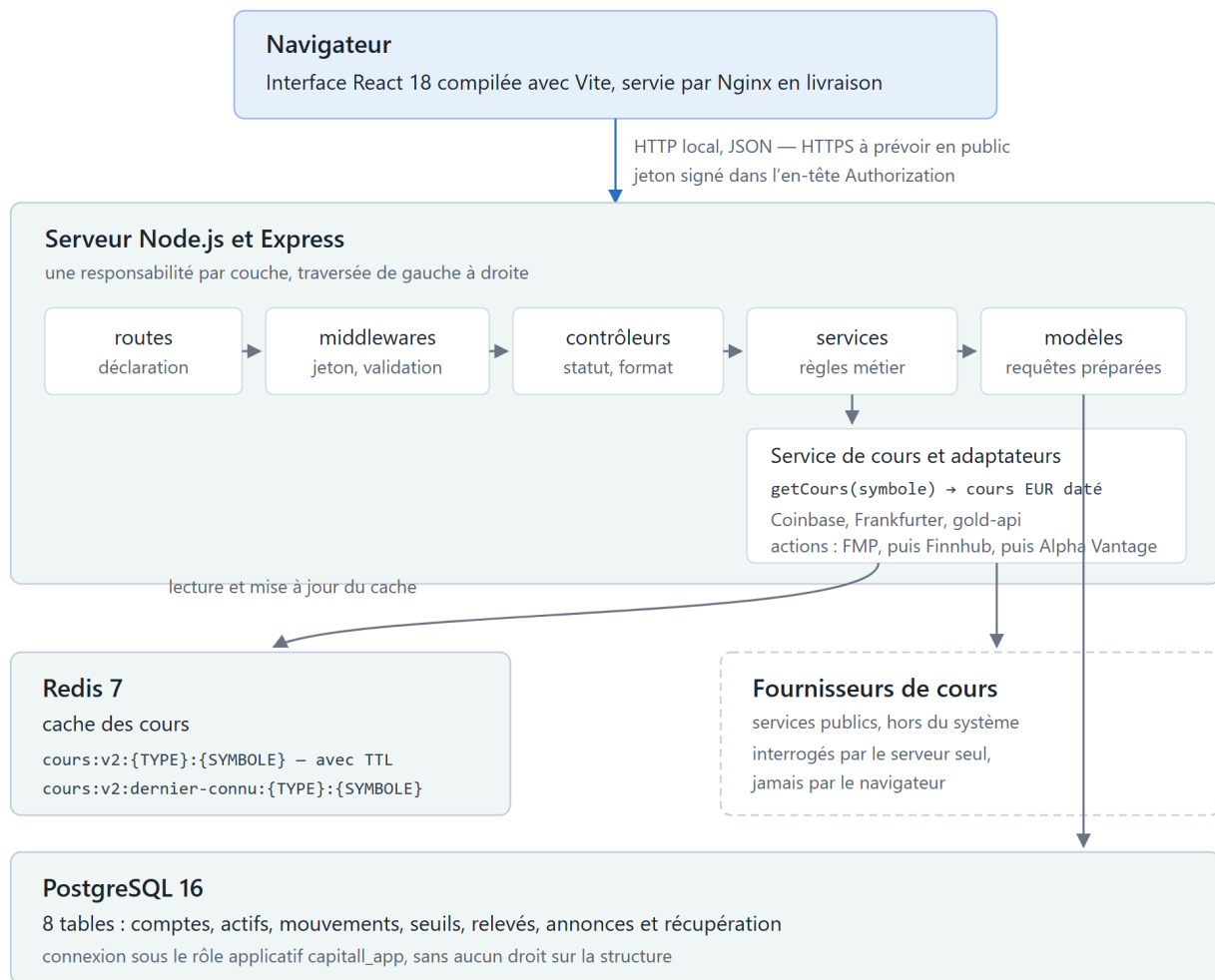


Figure 5 : Architecture de WalletWatch. Le navigateur ne parle qu'à l'API ; seul le serveur interroge les fournisseurs de cours et alimente le cache.

Organisation du serveur

Les routes déclarent les adresses et les contrôles d'accès. Des fonctions intermédiaires vérifient le jeton et les données reçues. Les contrôleurs préparent la réponse HTTP, les services appliquent les règles du projet et les modèles exécutent les requêtes SQL.

Le moteur de calcul reçoit une liste de mouvements et renvoie un résultat. Il peut donc être testé sans base de données ni serveur HTTP. Les tests remplacent au besoin l'accès aux données ou aux cours par des réponses maîtrisées. Le corps du dossier présente les extraits utiles ; l'annexe D conserve deux compléments courts.

Chaque classe d'actif possède un adaptateur de cours. Tous présentent une méthode `getCours(symbole)` et rendent un format commun. Les différences de champs, de devises et d'erreurs sont traitées à ce niveau, avant les calculs.

Répartition des responsabilités

Le **serveur calcule** les quantités, prix de revient, plus-values, répartitions et seuils. L'interface met ces résultats en forme, choisit la plage visible d'une courbe et trie les éléments reçus. Les coordonnées d'un graphique peuvent utiliser des nombres JavaScript : elles ne servent pas à prendre une décision métier.

La conversion d'affichage entre l'euro et le dollar utilise un taux fourni par le serveur et des fonctions de calcul décimal. Elle ne modifie ni les mouvements enregistrés ni leur prix de revient. Un même jeu de données vérifie le formatage côté serveur et côté interface.

Chargement du patrimoine

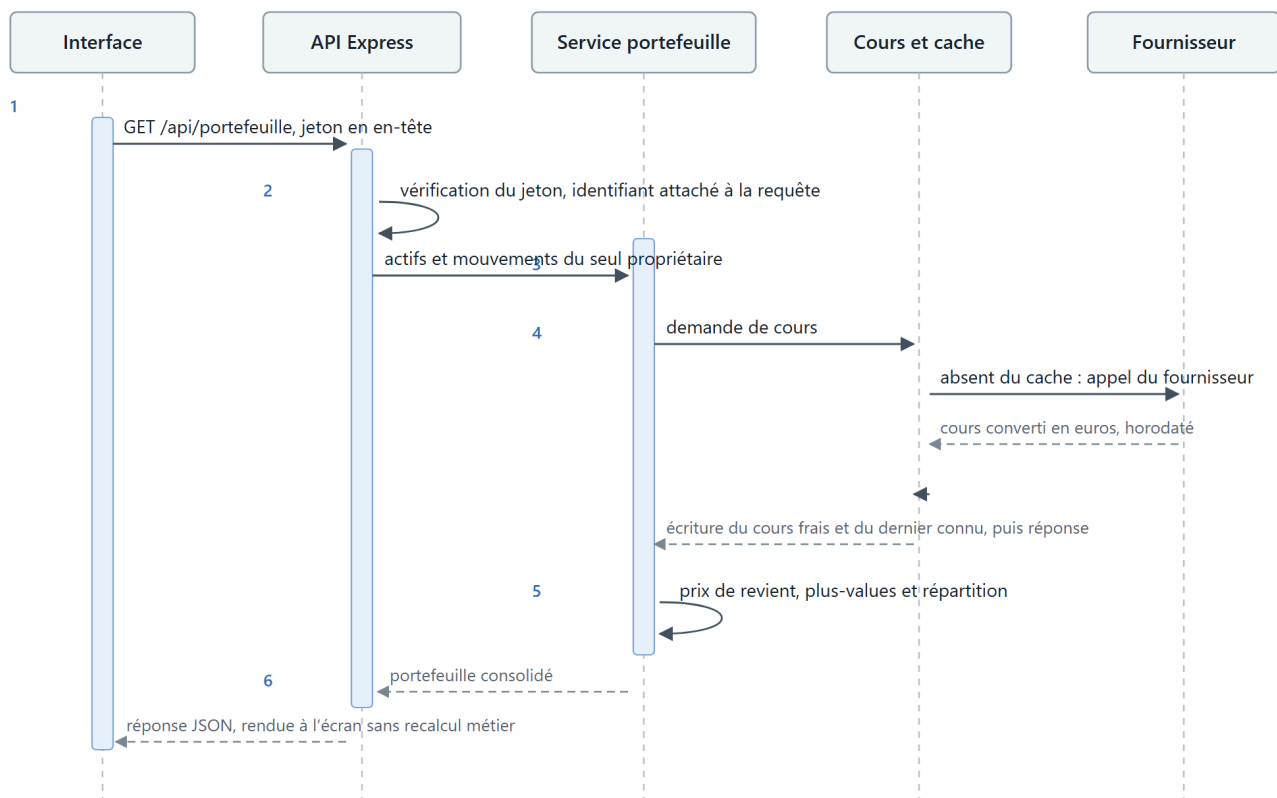


Figure 6 : Chargement du tableau de bord, du premier appel jusqu'au rendu. Le cours n'est demandé au fournisseur que s'il manque au cache.

Le diagramme représente le chemin de consultation. La colonne « Cours et cache » regroupe le service de cours et son accès Redis ; Redis ne contacte pas lui-même les fournisseurs. Les cours frais sont cherchés dans Redis, puis auprès des fournisseurs si nécessaire. En cas d'échec, le dernier cours connu peut être rendu avec sa date.

Le chargement de l'écran Patrimoine déclenche aussi `POST /api/portefeuille/actualisation`, qui tente d'enregistrer les relevés et d'évaluer les seuils. Le `GET /api/portefeuille` ne réalise pas ces écritures dans PostgreSQL ; il peut néanmoins alimenter Redis lorsqu'il récupère un cours. Les deux comportements ne doivent donc pas être confondus avec une absence de toute écriture.

Les écritures de relevés ne bloquent pas l'affichage si elles échouent. Un historique affiché peut ainsi être plus ancien que la valorisation courante. Cette limite doit rester visible dans la date du dernier relevé.

2.3 Environnement de développement et outillage

J'ai travaillé sous Windows avec Visual Studio Code, Git, Node.js et Docker. Le dépôt rassemble `backend/`, `frontend/` et `docs/`, avec les fichiers de composition à la racine. La version Node du poste contrôlée le 10 septembre est 24.13.0 ; les images du projet utilisent la branche 22.

En développement, PostgreSQL et Redis tournent dans des conteneurs, tandis que l'API et Vite peuvent fonctionner sur le poste avec rechargement automatique. La pile de livraison comprend les quatre services. Pour les essais avec une vraie base, des noms de projet, ports et volumes distincts évitent de toucher aux données de travail.

Les commandes npm sont déclarées dans les fichiers `package.json`. Elles permettent de lancer l'application, les tests, l'analyse statique et la construction. Les fichiers de verrouillage consignent les versions installées. Les modèles d'environnement décrivent la configuration attendue ; les valeurs réelles

restent hors des fichiers à publier.

Le serveur vérifie sa configuration dès le démarrage. Cela permet de signaler une variable obligatoire manquante avant de recevoir une requête. La procédure de déploiement complète est présentée en 3.11 et dans `docs/déploiement.md`.

J'ai consulté les documentations techniques en anglais lorsque le comportement d'un outil le demandait, notamment pour les réponses des fournisseurs, les verrous PostgreSQL et les attributs d'accessibilité. Les choix retenus sont ensuite expliqués dans le code ou la documentation du projet.

Partie 3 : Conception et réalisation du projet

3.1 Maquettage des interfaces

Préparer une présentation commune

J'ai préparé dans Figma une palette sombre, la typographie, les espacements et les composants réutilisables avant d'intégrer les écrans. L'objectif était de conserver les mêmes repères pour les boutons, champs, classes d'actifs et variations. Le thème sombre est un choix du projet ; il ne dispense pas de vérifier le contraste des textes.

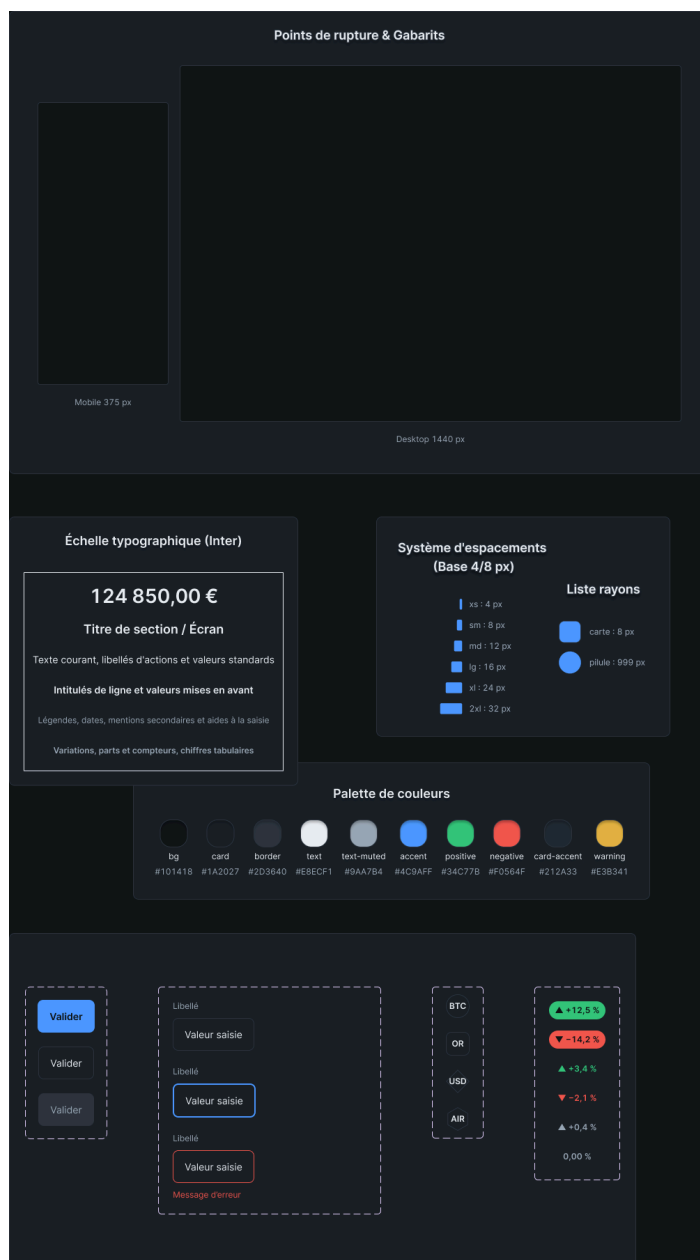


Figure 7 : Le système de design : points de rupture, échelle typographique, système d'espacements, palette et composants maîtres avec leurs variantes.

Cette planche présente le système de conception historique. Les maquettes conservent le nom CapitAll. L'interface livrée s'appelle WalletWatch et certains détails ont évolué pendant l'intégration. Les variables CSS et les composants actuels sont la référence pour le rendu final.

Adaptations téléphone et bureau

Sept écrans ont été maquettés à 375 pixels : connexion, patrimoine, positions, détail d'une position, mouvement, seuils et compte. Les vues Patrimoine, Positions et Détail ont aussi été dessinées à 1440 pixels, car leur structure change davantage sur un grand écran.

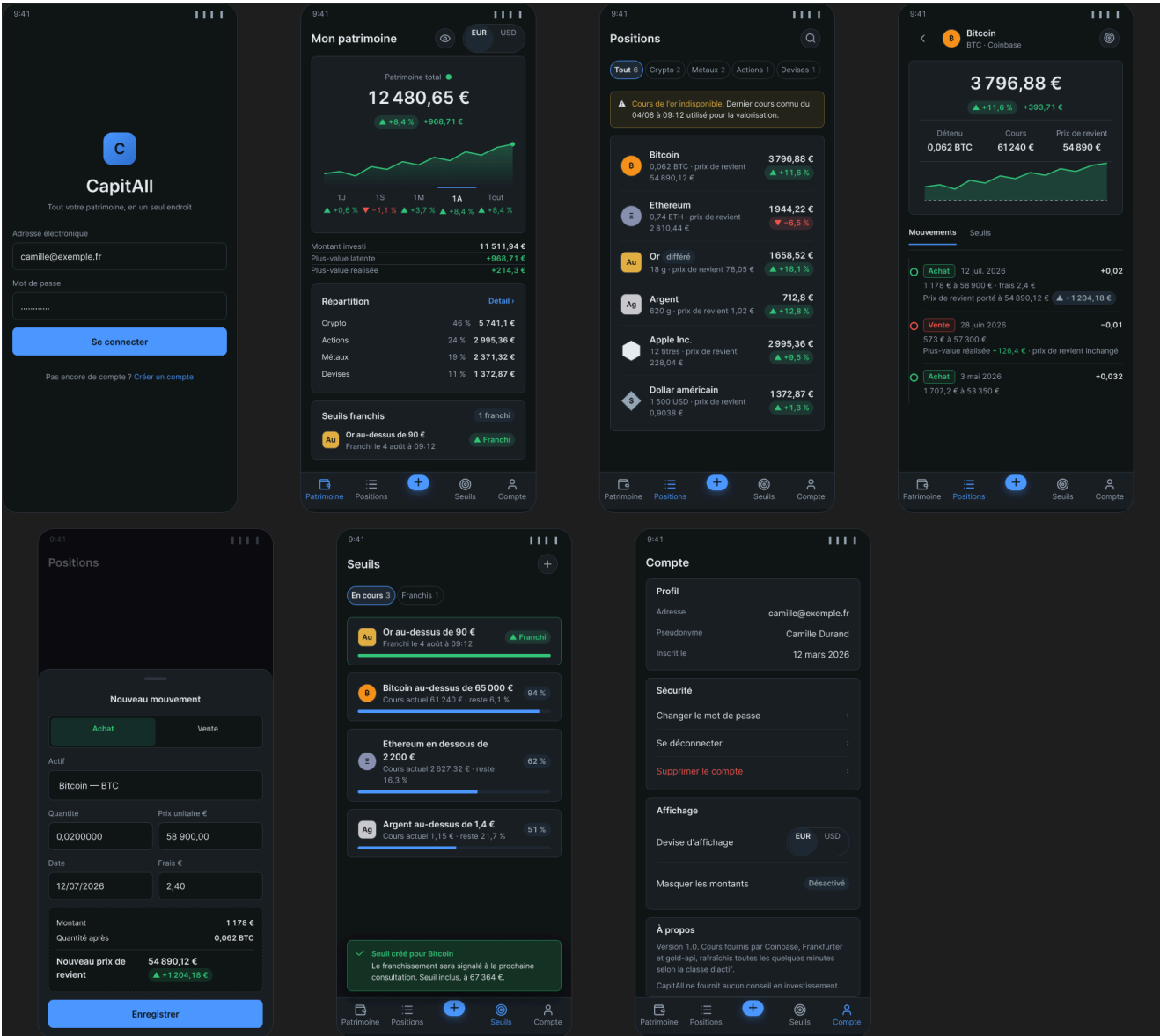


Figure 8 : Les sept écrans maquettés en 375 px, présentés sur deux rangées. Le détail de chaque écran figure en annexe A.

Sur téléphone, la navigation principale se trouve en bas et les positions sont présentées sous forme de cartes. Sur bureau, la navigation devient latérale et les positions utilisent un tableau. Le formulaire de mouvement reste lié à l'écran depuis lequel il est ouvert. Les planches agrandies et les trois maquettes bureau sont regroupées en annexe A ; l'enchaînement est présenté en 1.3.

Prévoir les états et les évolutions

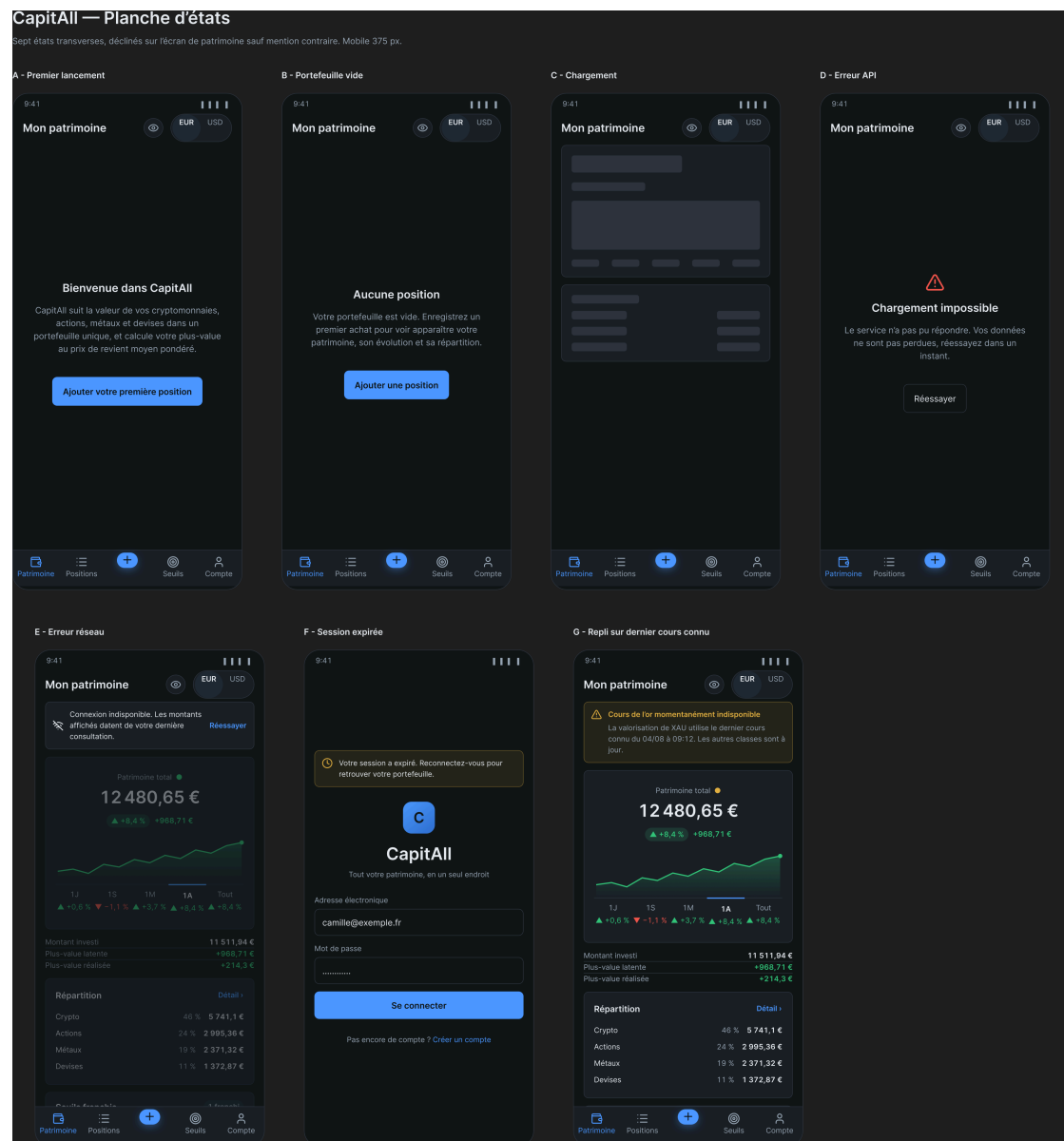


Figure 9 : La planche d'états : premier lancement, portefeuille vide, chargement, erreur d'API, erreur réseau, session expirée et dernier cours connu.

La conception ne se limite pas aux écrans remplis. J'ai prévu le chargement, le portefeuille vide, les erreurs d'API et de réseau, la session expirée et l'emploi du dernier cours connu. Les messages précisent ce qui manque et proposent une action lorsqu'un nouvel essai est possible.

Les couleurs des gains et pertes sont accompagnées d'un signe et d'un libellé. La répartition possède une liste chiffrée en complément de l'anneau. Les zones interactives, l'ordre de lecture et le focus sont ensuite vérifiés sur le navigateur : une maquette ne prouve pas le fonctionnement au clavier.

Les captures récentes montrent des ajouts postérieurs aux premières maquettes : sortie sans vente, correction d'un mouvement, choix de l'unité des frais et récupération de compte. Je conserve les maquettes comme traces de conception et j'explique ces écarts au lieu de les présenter comme une reproduction exacte du produit livré.

3.2 Réalisation des interfaces statiques

De la maquette au code

J'ai regroupé les couleurs et les espacements communs dans un fichier de variables CSS pour limiter les écarts entre écrans. Les composants réutilisent ces valeurs. Certaines couleurs propres aux classes d'actifs et à des éléments locaux restent définies dans leurs composants ; toutes les valeurs du projet ne sont donc pas centralisées.

`frontend/src/styles/tokens.css` : la palette et le système d'espacement, en un seul endroit.

```
:root {
  --couleur-fond: #101418;          --couleur-carte: #1a2027;
  --couleur-bordure: #2d3640;       --couleur-texte: #e8ecf1;
  --couleur-texte-attenue: #9aa7b4; --couleur-accent: #4c9aff;
  --couleur-positif: #34c77b;
  /* Ajusté de #F0564F à #F8645D : 4,83:1 sur la carte haute #212A33. */
  --couleur-negatif: #f8645d;

  --espace-1: 4px; --espace-2: 8px; --espace-3: 12px;
  --espace-4: 16px; --espace-6: 24px; --espace-8: 32px;

  --rayon-carte: 8px; --rayon-pilule: 999px;
}
```

Le commentaire au-dessus du rouge documente une correction plutôt qu'un choix initial : la direction artistique retenait #F0564F, qui tombait sous le seuil de contraste une fois mesuré sur le fond de la carte accentuée. Il a été éclairci jusqu'à #F8645D. La valeur écartée est restée dans le commentaire : cela explique la différence avec la couleur initialement choisie.

L'échelle d'espacement est délibérément courte, six valeurs multiples de 4, et trouée : il n'y a pas de --espace-5 ni de --espace-7. Cette liste limitée m'aide à conserver des espacements cohérents.

Un socle, puis des composants

Le fichier de socle applique une remise à zéro minimale et les variables communes. Il définit aussi l'indicateur `:focus-visible` pour tout le document, avec un contour de 2 px et un décalage de 2 px. Le focus reste ainsi visible lors d'une navigation au clavier.

Le reste de l'interface est découpé en composants qui portent chacun leur feuille de style : carte, bouton, champ, pilule de variation, jeton de classe, courbe, feuille glissante, tableau de positions. Les composants partagent les variables communes, avec quelques valeurs locales liées à leur usage.

L'accessibilité comme contrainte de construction

J'ai pris en compte les libellés, les erreurs, le focus et les informations qui ne doivent pas dépendre uniquement d'une couleur. Il s'agit de mesures concrètes d'accessibilité, sans revendication de conformité RGAA complète.

Le composant `Champ` associe le libellé, le message d'aide et l'erreur à la saisie. Cette structure commune évite de refaire ces associations dans chaque écran. Le fichier complet reste disponible dans `frontend/src/composants/Champ.jsx`.

L'astérisque visuel du champ obligatoire est accompagné d'un texte destiné aux technologies d'assistance. `aria-invalid` signale une saisie invalide, `aria-describedby` relie l'aide et l'erreur, et `useId` fournit un identifiant propre au composant.

Les classes d'actifs sont aussi nommées en texte ; leur compréhension ne repose pas uniquement sur une couleur ou une forme.

Les vues actuelles de `Positions`, en annexe E, montrent les signes et les flèches associés aux variations. La couleur est un repère supplémentaire. Les contrôles de contraste réalisés pendant le projet sont ciblés ; ils ne constituent pas un audit RGAA complet.

La règle CSS conserve un indicateur visible lorsqu'un élément reçoit le focus au clavier. Les dialogues gèrent le focus et la touche Échap. Les vérifications ciblées du projet ne permettent pas de revendiquer une conformité globale de l'accessibilité.

Une interface adaptée aux différentes largeurs

Le principal changement de disposition se fait à 768 px. Sous ce seuil, la navigation devient une barre basse et les formulaires s'ouvrent en feuille glissante. Sur bureau, la navigation est latérale et les dialogues sont centrés. Des ajustements à d'autres largeurs complètent cette règle.

Trois écrans seulement changent réellement de structure entre les deux : le patrimoine passe d'une colonne à deux, le tableau des positions se transforme en pile de cartes, et le détail d'une position réorganise ses trois blocs de tête. Les autres se contentent d'une largeur maximale centrée. Quelques ajustements ponctuels s'ajoutent à cette règle principale, à 480 px pour les très petits téléphones et à 900 px pour la largeur du rail, mais ils ne changent jamais la disposition, seulement des dimensions.

Le résultat rendu par l'application est présenté en 3.3, où la partie dynamique donne aux écrans leur contenu réel.

3.3 Réalisation des interfaces dynamiques

Un seul point de passage vers le serveur

Les appels à l'API sont regroupés dans un module commun. Il ajoute l'autorisation, traduit les erreurs et permet de traiter la perte de session de façon cohérente entre les écrans.

`frontend/src/services/api.js` : le point de passage unique ; l'extrait ci-dessous montre le traitement commun.

```
export async function requete(chemin, { methode = 'GET', corps, jeton } = {}) {
  // [...] en-têtes, dont Authorization si un jeton est fourni
  let reponse;
  try {
    reponse = await fetch(`${BASE}${chemin}`, { /* [...] */ });
  } catch {
    // Coupure réseau ou serveur injoignable : l'appelant reçoit un message lisible
    // plutôt qu'une exception technique.
    throw new ErreurApi('Le serveur est injoignable. Vérifiez votre connexion.', 0);
  }

  const donnees = await lireCorps(reponse);
  if (reponse.ok) return donnees;

  if (reponse.status === 401) {
    // La session n'est plus valable : le contexte vide le jeton, la garde de routes
    // renvoie alors vers la connexion.
    surSessionPerdue?();
  }

  throw new ErreurApi(donnees?.erreur ?? 'Une erreur est survenue.',
    reponse.status, donnees?.champs);
}
```

Ce module conserve le message d'erreur renvoyé par le serveur, attribue le statut conventionnel 0 à une coupure réseau sans réponse HTTP et utilise un chemin relatif pour l'API. Les écrans peuvent ainsi distinguer un refus du serveur d'une absence de réponse, sans compiler d'adresse propre à un environnement.

La session et la garde de routes

Le jeton d'authentification vit dans le stockage de session du navigateur : il survit au rechargement et à la navigation, et est normalement supprimé à la fermeture de l'onglet ; une restauration de session du navigateur peut modifier ce comportement. Le stockage local reste écarté, rien ne doit rester derrière

l'utilisateur sur un poste partagé, et le cookie aussi, pour la raison exposée en 3.7.

Au départ, le jeton était seulement conservé dans l'état React : recharger la page déconnectait l'utilisateur. Le stockage de session améliore ce comportement, mais reste accessible au JavaScript de la page. La politique de sécurité du contenu et la révocation réduisent certains risques ; elles ne garantissent pas l'absence de vol de jeton.

Lorsqu'une réponse porte le statut 401, le client d'API prévient le contexte d'authentification. Celui-ci vide la session et conserve un indicateur distinct d'une déconnexion volontaire, afin d'expliquer le retour à l'écran de connexion.

La garde de routes complète le dispositif. Elle mémorise la route demandée dans l'état de navigation, ce qui permet d'y revenir après connexion plutôt que d'atterrir systématiquement sur le tableau de bord ; c'est d'autant plus utile qu'un jeton peut expirer au milieu d'une consultation.

Ce que l'interface calcule, et ce qu'elle ne calcule pas

La règle posée en 2.2 se vérifie ici. L'interface ne recalcule ni le prix de revient, ni la plus-value, ni la répartition, ni l'écart avant un seuil. Elle reçoit ces valeurs du serveur et les met en forme. La conversion d'affichage entre l'euro et le dollar utilise des fonctions décimales vérifiées avec le même jeu d'essai des deux côtés.

Le récapitulatif de saisie d'un mouvement illustre la règle mieux qu'un raisonnement. Avant d'enregistrer quoi que ce soit, l'écran demande au serveur ce que l'opération produirait : quantité détenue après, nouveau prix de revient, écart, plus-value dégagée s'il s'agit d'une vente. Rien n'est écrit en base ; le moteur rejoue l'historique de la position avec le mouvement hypothétique en plus.

WalletWatch

Bitcoin Renommer

BTC · Cryptos · four

Patrimoine

Positions

Seuils

Compte

Session ouverte
Camille Durand

Se déconnecter

Nouveau mouvement

SENS DE L'OPÉRATION

Achat Vente

Sortie

ACTIF *

Bitcoin — BTC (Cryptos)

Cryptos [Suivre un nouvel actif](#)

QUANTITÉ * PRIX UNITAIRE (€) *

0,2 58900

① Cours du jour : 67 042,62 €
Conservez-le pour une opération du moment, corrigez-le pour une opération passée.

DATE DE L'OPÉRATION * FRAIS (€)

10/09/2026 2,40

Facultatif. Les frais d'achat entrent dans le prix de revient.

CES FRAIS ONT ÉTÉ PRÉLEVÉS EN

euros

EFFET DE CE MOUVEMENT

Montant de l'opération	11 780 €
Frais	2,4 €
Quantité détenue après	0,8 BTC
Nouveau prix de revient	57 220,19 € +563,94 €

Annuler Enregistrer

Valorisation de la position

40 225,57 €

GAINS LATENTS

▲ +8 231,82 €

Jour

68997€

67586€

63656€

60156€

Prix de revient 56 656,25 €

2026-08-11 2026-08-17

Mouvements 3 Seuils

Vente 17 août 2026

Prix unitaire 63 500 €

Effet sur le prix de revient inchangé

Corriger Supprimer

Achat 16 juillet 2026

Prix unitaire 61 000 €

Corriger Supprimer

Achat 10 juin 2026

0,6 BTC

67 042,62 €

56 656,25 €

Année

▲ +19,5 %

Suivi

▲ +19,5 %

2026-08-29 2026-09-01 2026-09-04 2026-09-10

-0,2 BTC

Plus-value réalisée +1 380,75 €

+0,3 BTC

Effet sur le prix de revient +2 626,25 €

+0,5 BTC

Figure 10 : Simulation d'un achat de 0,2 BTC : prix saisi de 58 900 €, frais de 2,40 € et récapitulatif calculé par le serveur. Aucun enregistrement. Capture du 10 septembre 2026.

La capture montre le récapitulatif d'un achat de 0,2 BTC à 58 900 €, avec 2,40 € de frais. Le nouveau prix de revient de 57 220,19 € et l'écart de +563,94 € sont calculés par le serveur. J'ai évité de refaire ce calcul dans le navigateur pour ne pas maintenir deux versions de la même règle. La section 3.8 vérifie ce résultat à la main.

Les états prévus pour l'interface

Avant le développement, j'ai maqueté sept états possibles pour les écrans qui appellent l'API. Cela m'a permis de prévoir le chargement, l'absence de données et les erreurs en plus du fonctionnement normal.

État	Traitement
chargement	squelette occupant la place exacte du contenu attendu, pas de rotateur
portefeuille vide	invitation à enregistrer un premier mouvement, avec l'action à portée
erreur d'API	message rendu par le serveur, action de réessai
erreur réseau	message distinct, reconnaissable au statut 0
session expirée	retour à la connexion, avec la raison affichée
dernier cours connu	valeur affichée avec sa date et une pastille ambre
position sans cours	tiret plutôt que zéro, et exclusion explicite du total

Lorsqu'aucun cours n'est disponible, la position n'est pas comptée pour zéro : elle est exclue du total et l'écran l'indique.

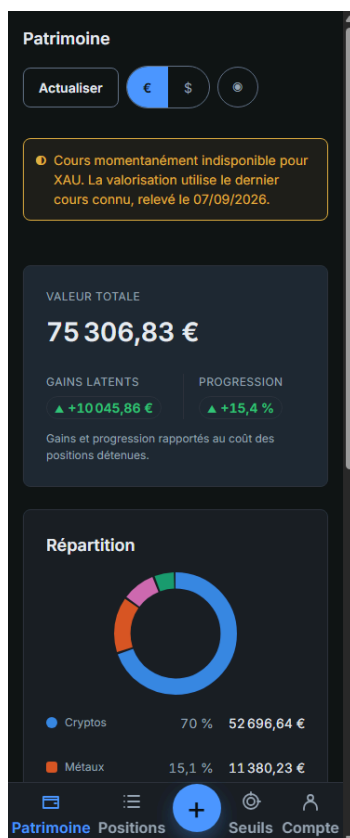
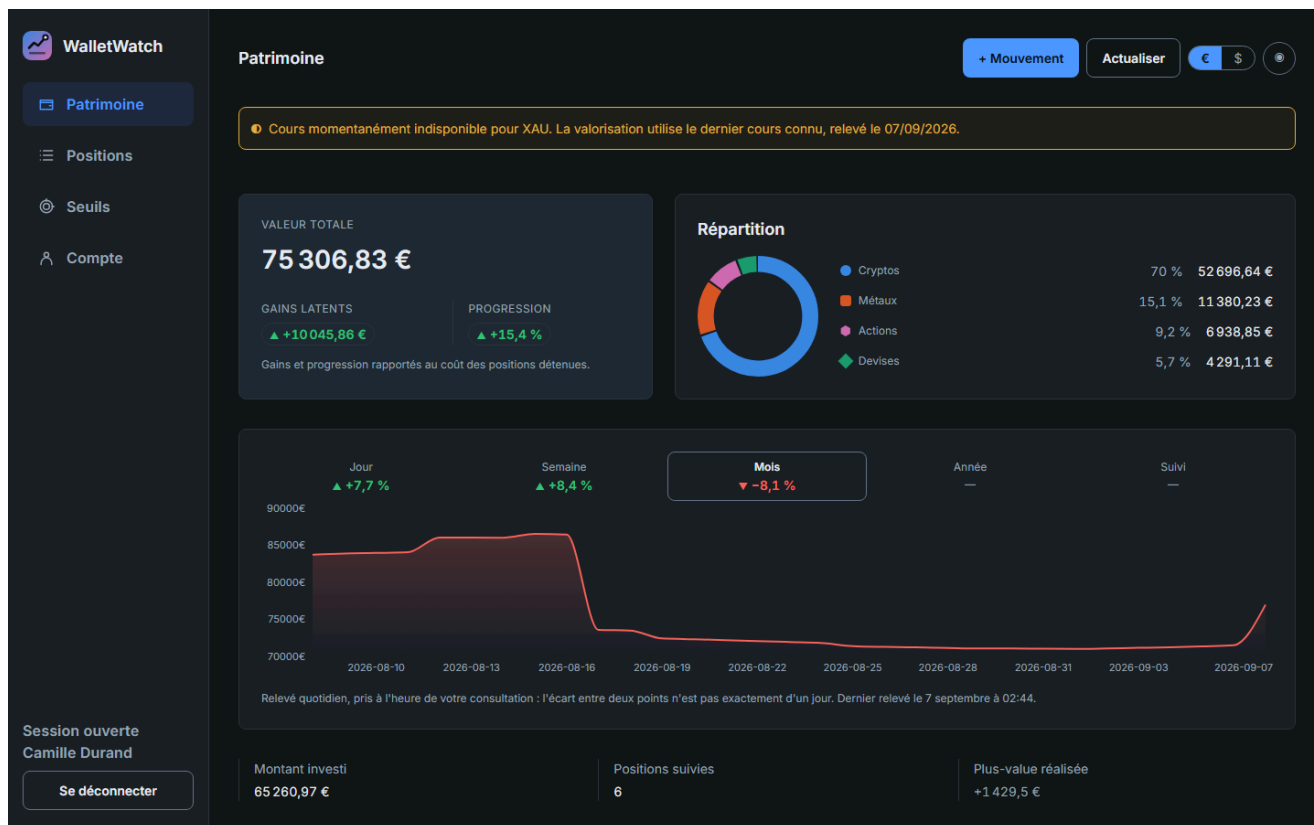


Figure 11 : Patrimoine sur bureau puis sur téléphone. Le bandeau signale le recours au dernier cours connu pour l'or. Captures du 10 septembre 2026.

Sur ces captures, le bandeau concerne l'or : un cours daté remplace temporairement le cours frais. Cette position reste valorisée avec une réserve visible. Ce cas est différent d'une position sans aucun cours, qui serait exclue du total. Sous la courbe, la date du dernier relevé est celle du 7 septembre : le point de la

journée n'était pas encore enregistré au moment de la capture, et l'affichage de la valeur courante n'en dépend pas (voir 2.2). Les vues bureau et mobile présentent les mêmes rubriques ; la partie basse se consulte par défilement sur téléphone.

Un détail de performance mérite d'être signalé parce qu'il touche à l'usage réel sur mobile : la courbe d'évolution est chargée à la demande, la bibliothèque de tracé pesant plus lourd que tout le reste de l'application réunie. Un squelette occupe sa place pendant ce temps.

Le sélecteur de plage relève du même souci, et il a été corrigé tard : chaque changement de plage rechargeait le portefeuille entier, cinq clics d'exploration valant cinq consolidations complètes alors que seul l'affichage changeait. La série arrive maintenant entière et le découpage se fait dans le navigateur, en jours et non en nombre de points : la série est trouée par construction, un relevé n'existant que si l'utilisateur a consulté ce jour-là.

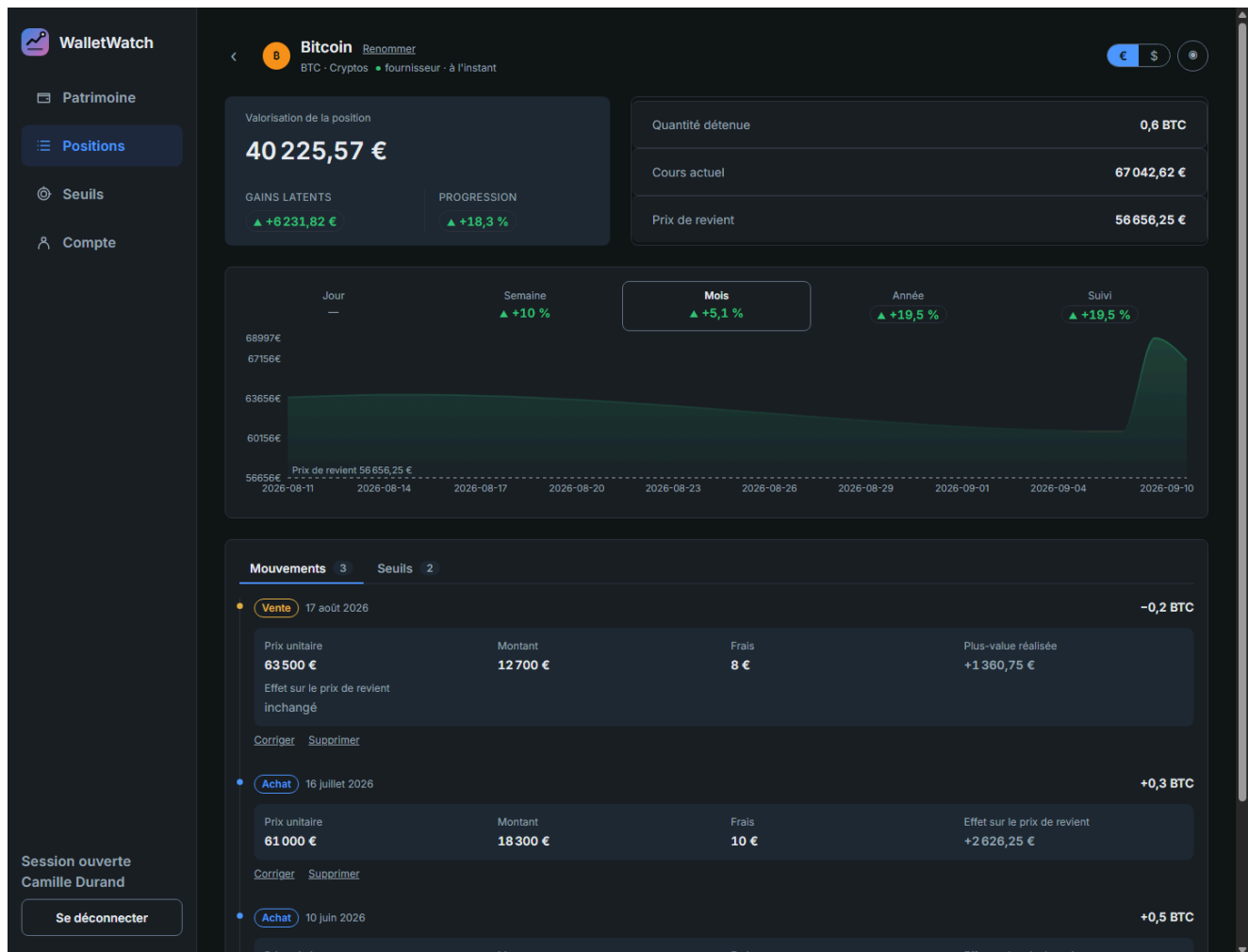


Figure 12 : Détail de Bitcoin : prix de revient de 56 656,25 € et vente du 17 août avec une plus-value réalisée de 1 360,75 €. Capture du 10 septembre 2026.

L'écran de détail est celui qui exploite le plus la partie dynamique. Le graphe y porte la ligne pointillée du prix de revient, tracée à partir d'une valeur calculée par le serveur, et l'aire entre la courbe et cette ligne est teintée selon le signe. En dessous, la frise des mouvements indique pour chacun son effet sur le prix de revient : la vente du 17 août affiche une plus-value réalisée de 1 360,75 € et la mention « inchangé », qui traduit la règle selon laquelle une vente ne déplace pas le prix de revient des titres restants. Cette mention est elle aussi une valeur du serveur, et non une déduction de l'interface.

L'écran des seuils applique le même partage : le serveur calcule la valeur observée et l'écart restant, puis l'interface les affiche avec une barre de progression. Le texte et les valeurs permettent de comprendre l'état du seuil sans s'appuyer uniquement sur sa couleur.

3.4 Mise en place de la base de données relationnelle

De la conception aux tables

J'ai distingué le modèle conceptuel, qui décrit les données et leurs relations, le modèle logique, qui les traduit en relations, puis le modèle physique créé dans PostgreSQL. Le diagramme donne une vue d'ensemble ; la liste des attributs figure dans le modèle logique ci-dessous et le SQL complet est en annexe B.

Le modèle conceptuel

Le diagramme présente les sept entités de la première modélisation, dont les annonces reportées. La récupération de mot de passe ajoute une huitième table technique au schéma physique. Le tableau ci-dessous complète le diagramme par les données portées par les entités ; le modèle logique donne ensuite les colonnes exactes.

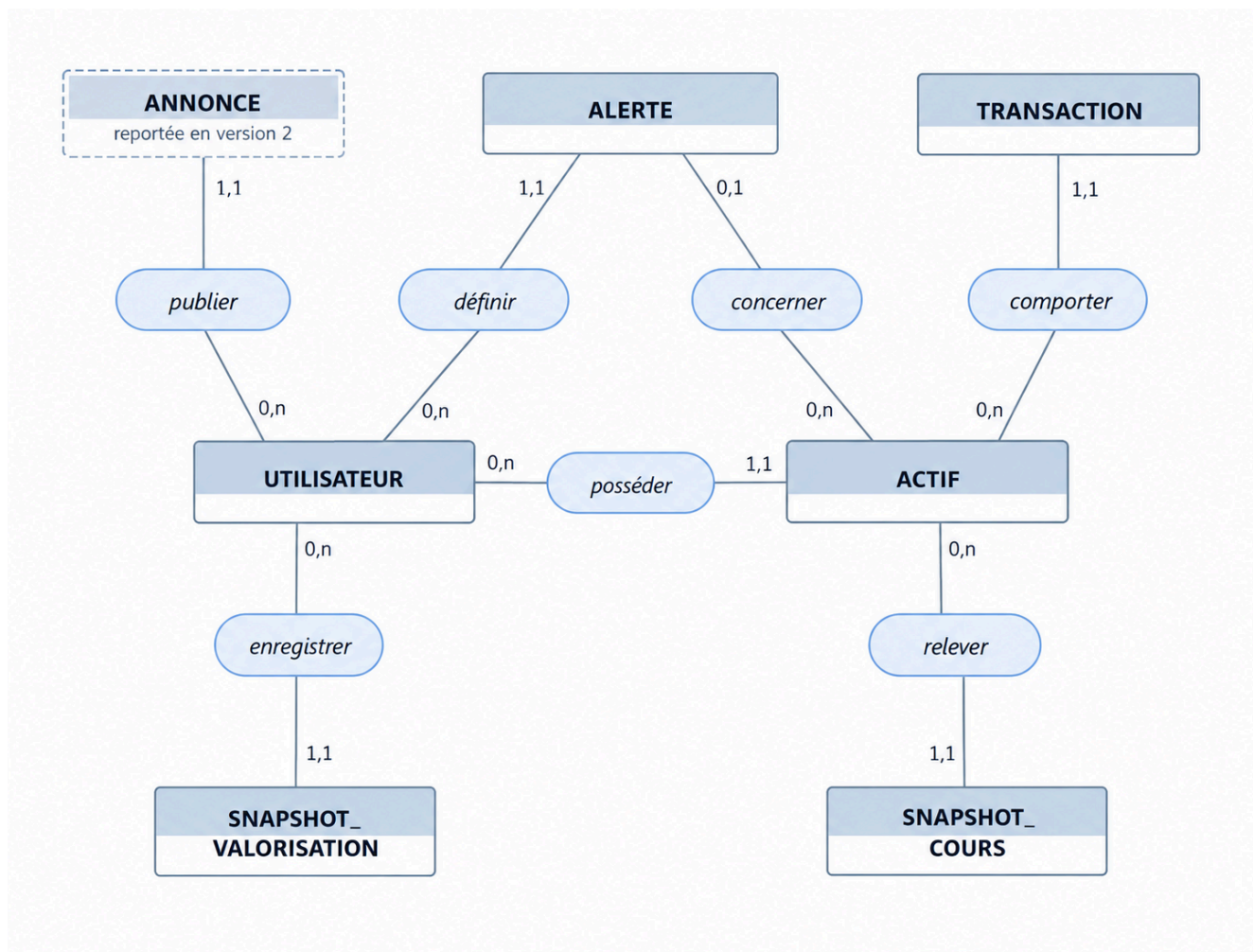


Figure 13 : Modèle conceptuel de données. La table d'annonces reste signalée comme une fonction reportée.

Les cardinalités traduisent plusieurs décisions de conception. Un compte neuf peut ne posséder aucun actif et un actif peut exister sans mouvement. Les seuils et les instantanés restent facultatifs tant que les fonctions associées n'ont pas été utilisées. Chaque entité enfant appartient en revanche à un parent. Seule exception : un seuil visant le patrimoine total ne référence aucun actif, ce qu'exprime la cardinalité 0,1.

Données du modèle conceptuel

Entité	Données principales	Relations
Utilisateur	Adresse, empreinte du mot de passe, pseudonyme, rôle, état, date d'inscription	Possède des actifs, seuils et relevés de valorisation
Actif	Classe, symbole, nom, date d'ajout	Appartient à un utilisateur ; possède des mouvements et des relevés de cours
Mouvement	Sens, quantité, prix, frais et unité, date, note	Concerne exactement un actif
Seuil	Cible, sens, valeur, état, dates de création et de franchissement	Appartient à un utilisateur ; peut cibler un actif ou le patrimoine total
Relevé de valorisation	Jour, heure et valeur totale en euros	Concerne un utilisateur
Relevé de cours	Jour, heure, cours en euros et quantité détenue	Concerne un actif
Annonce, reportée	Titre, contenu, épinglage, date de publication	Possède un auteur

Une demande de réinitialisation relie en plus un utilisateur à une empreinte de clé, une date d'expiration et une date d'utilisation éventuelle. Ce support technique est représenté dans le modèle physique à huit tables.

Éléments non stockés dans le modèle

J'ai retenu les limites de modélisation suivantes.

Le cours courant n'est pas une entité. Sa valeur change rapidement et serait déjà périmée lors d'une consultation suivante. Il est donc conservé dans un cache technique, hors du modèle relationnel (voir 3.5).

Le prix de revient et les plus-values ne sont pas stockés. Ils sont recalculés à partir des mouvements à chaque consultation. Une correction de mouvement est ainsi prise en compte sans devoir synchroniser des résultats intermédiaires (voir 3.6).

Il n'y a pas de référentiel d'actifs partagé. J'ai envisagé une table de symboles commune à tous les comptes, puis je l'ai écartée : elle impose de gérer les conflits de nommage entre utilisateurs et n'apporte rien tant qu'aucune fonctionnalité ne compare les portefeuilles entre eux.

Les deux tables d'instantanés constituent une exception. La valeur d'un portefeuille ou le cours d'un actif à une date passée ne peuvent pas toujours être recalculés : les fournisseurs gratuits ne proposent pas tous le même historique, et l'un d'eux n'en fournit aucun. Chaque instantané conserve donc une valeur réellement relevée à une date donnée.

L'instantané de valorisation référence l'utilisateur, car il concerne tout son portefeuille. Le relevé de cours référence seulement l'actif, qui permet déjà de retrouver son propriétaire. J'ai évité de dupliquer cet identifiant dans les deux relations.

Le modèle logique

En notation relationnelle, clés primaires préfixées par # et clés étrangères par -> :

```
utilisateur (#id, email UNIQUE, mot_de_passe_hache, pseudo, role, actif,
            jetons_invalides_avant, date_inscription)
actif       (#id, ->utilisateur_id, type, symbole, nom, date_ajout,
            UNIQUE(utilisateur_id, symbole))
transaction (#id, ->actif_id, sens, quantite, prix_unitaire, frais, frais_montant,
            frais_unite, date_transaction, note)
alerte      (#id, ->utilisateur_id, ->actif_id NULL, type_cible, sens_seuil,
            valeur_seuil, statut, date_creation, date_declenchement)
snapshot_valorisation (#id, ->utilisateur_id, date_snapshot, valeur_totale_eur,
```

```

    heure_releve, UNIQUE(utilisateur_id, date_snapshot))
snapshot_cours (#id, ->actif_id, date_snapshot, cours_eur, quantite, heure_releve,
    UNIQUE(actif_id, date_snapshot))
annonce      (#id, ->auteur_id, titre, contenu, epinglee, date_publication)
reinitialisation_mot_de_passe (#id, ->utilisateur_id, jeton_hache UNIQUE,
    date_creation, expire_le, utilise_le)

```

Les contraintes composées appliquent deux règles du projet. `UNIQUE(utilisateur_id, symbole)` empêche un compte de suivre deux fois le même symbole. `UNIQUE(utilisateur_id, date_snapshot)` limite la valorisation à une ligne par compte et par jour, quel que soit le nombre de consultations.

Le modèle physique

Le script de création fait foi ; ce qui suit en décrit les partis pris.

`backend/db/schema.sql` : la table des mouvements, où se concentrent les règles de domaine.

```

CREATE TABLE transaction (
    id                SERIAL PRIMARY KEY,
    actif_id          INTEGER NOT NULL REFERENCES actif(id) ON DELETE CASCADE,
    sens              VARCHAR(25) NOT NULL
                    CHECK (sens IN ('achat', 'vente', 'sortie_non_marchande')),
    quantite          NUMERIC(38, 18) NOT NULL CHECK (quantite > 0),
    prix_unitaire     NUMERIC(38, 18) NOT NULL CHECK (prix_unitaire >= 0),
    frais             NUMERIC(18, 2) NOT NULL DEFAULT 0 CHECK (frais >= 0),
    frais_montant     NUMERIC(38, 18) NOT NULL DEFAULT 0 CHECK (frais_montant >= 0),
    frais_unite       VARCHAR(20) NOT NULL DEFAULT 'EUR',
    date_transaction  TIMESTAMPTZ NOT NULL,
    note              TEXT,
    CONSTRAINT transaction_frais_euros_coherents
        CHECK (frais_unite <> 'EUR' OR frais_montant = frais),
    CONSTRAINT transaction_sortie_sans_prix
        CHECK (sens <> 'sortie_non_marchande' OR prix_unitaire = 0)
);

```

Pour les colonnes financières, j'ai retenu un type exact plutôt que `DOUBLE PRECISION`, qui introduit une imprécision à chaque opération. J'ai aussi revu le nombre de décimales en cours de projet : huit suffisent au bitcoin, mais l'ether se divise en dix-huit, et deux décimales pouvaient arrondir à zéro un cours inférieur à un demi-centime. Quantités et prix sont donc en `NUMERIC(38, 18)` ; les montants en euros restent au centime.

Les frais utilisent trois colonnes. Si une plateforme retient 0,002 ETH, `frais_montant` et `frais_unite` conservent le prélèvement d'origine, tandis que `frais` en garde la contre-valeur en euros utilisée par le moteur. Deux contraintes vérifient que le montant et sa contre-valeur sont identiques lorsque l'unité est l'euro, et qu'une sortie non marchande ne possède aucun prix de vente.

`ON DELETE CASCADE` figure sur les clés étrangères de dépendance : la suppression d'un compte atteint en un saut ses actifs, ses seuils, ses instantanés et ses annonces, et en deux sauts ses mouvements, ses relevés de cours et les seuils qui ciblent l'un de ces actifs. C'est la base qui assure que les données rattachées au compte sont supprimées de la base active, et non un enchaînement d'appels applicatifs qu'on pourrait oublier de compléter.

La table des seuils porte une contrainte `CHECK` entre `type_cible` et `actif_id`. Elle interdit un seuil visant un actif sans le référencer, ainsi que le cas inverse. La base peut garantir cette règle, car elle concerne deux colonnes de la même ligne.

Répartition des contrôles entre la base et le code. La règle « on ne vend pas plus que ce que l'on détient » n'apparaît nulle part dans le script, et c'est volontaire : elle porte sur la somme de plusieurs lignes de mouvements, à chaque étape de leur ordre chronologique. Aucune contrainte `CHECK`, qui ne voit qu'une ligne, ne peut l'exprimer. Elle est vérifiée côté serveur, dans une transaction avec verrou (voir 3.5 et 3.6). La base contrôle les règles qui concernent une ligne ; le serveur traite celles qui demandent de relire plusieurs mouvements.

Sept index explicites complètent ceux que PostgreSQL crée pour les clés primaires et les contraintes d'unicité. `idx_alerte_actif` est partiel : il indexe seulement les lignes dont `actif_id` n'est pas nul. Les seuils visant le patrimoine total sont ainsi exclus, car aucune requête ne les recherche par identifiant d'actif. Les tables d'instantanés n'ont pas d'index supplémentaire : leur contrainte d'unicité en crée déjà un sur le couple propriétaire-date.

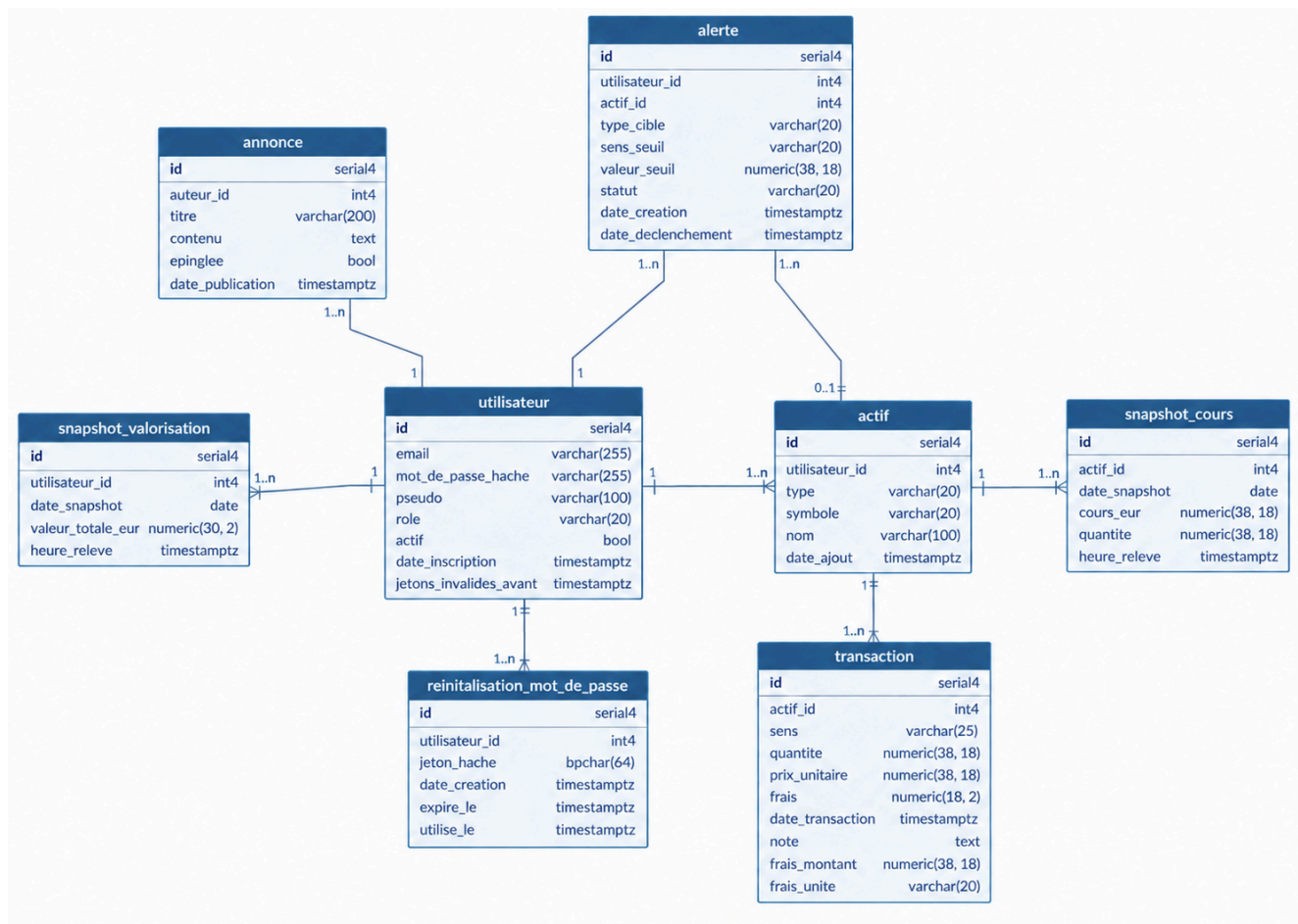


Figure 14 : Schéma relationnel relevé après exécution du script : huit tables, leurs colonnes et leurs relations.

La figure montre le schéma obtenu après exécution du script : huit tables, leurs colonnes typées et les liens réellement créés. Elle permet de comparer la conception avec la base effectivement utilisée.

Une neuvième table, `migration_appliquee`, sert de registre à l'outil de migration. Elle contient un nom de fichier, une empreinte et une date, mais aucune donnée métier ; je ne l'ai donc pas ajoutée au modèle du domaine.

Un rôle applicatif à droits restreints

L'application ne se connecte jamais avec le propriétaire de la base. Le rôle `capital1_app` peut utiliser le schéma, lire et modifier les données, et accéder aux séquences nécessaires aux colonnes SERIAL.

Le droit de connexion est accordé dans un bloc séparé, par format (`'GRANT CONNECT ON DATABASE %I ...'`, `current_database()`) : `GRANT` n'accepte pas de paramètre, et écrire le nom de la base en dur aurait obligé à modifier ce fichier pour toute base portant un autre nom. Les droits sur les séquences sont nécessaires aux colonnes SERIAL : sans eux, la première insertion échouerait.

Le rôle applicatif n'a aucun droit de création, de modification ou de suppression de structure. Une injection SQL qui parviendrait malgré tout à s'exécuter ne pourrait donc ni créer ni supprimer une table. Le mot de passe inscrit au script est une valeur de développement, remplacée à l'initialisation par une variable d'environnement.

Jeu de données et évolution du schéma

Le script de jeu de données crée trois comptes de démonstration et un portefeuille couvrant les quatre classes d'actifs. L'un des comptes est volontairement désactivé pour montrer le refus de connexion. Le script vide les tables avant de réinsérer les données : il peut donc être relancé pour repartir d'un état connu. Les mots de passe sont hachés dans la base par l'extension pgcrypto, avec le même algorithme et le même coût que côté serveur. J'ai écarté des empreintes recopiées, dont l'origine aurait été moins claire.

Une base déjà en service ne se recrée pas : elle se fait évoluer par migrations. Le dépôt en compte six, nommées par leur date d'application et rejouables sans double effet : l'une ajoute la table des relevés de cours, une autre élargit les colonnes numériques, une troisième introduit les frais en unité d'origine.

La base conserve la liste des migrations appliquées et l'empreinte de chaque fichier. L'outil refuse ainsi une migration déjà exécutée puis modifiée. Chaque script vérifie aussi son résultat avant d'être inscrit dans le registre. Je n'ai pas prévu de retour arrière automatique : réduire une échelle détruirait des décimales qu'un script inverse ne pourrait pas recréer.

J'ai rencontré un problème de droits : une table créée par migration n'héritait pas des autorisations du rôle applicatif, créé auparavant par le script initial. La migration accorde désormais ces droits explicitement. Sans cette correction, la première lecture de la nouvelle table aurait échoué.

La sauvegarde et la restauration passent par pg_dump et psql depuis le conteneur de la base ; la procédure figure en 3.11.

3.5 Composants d'accès aux données

PostgreSQL conserve les données métier. Redis stocke temporairement les cours, qui peuvent être rechargés si le cache disparaît.

Accès centralisé et requêtes paramétrées

Tous les modèles utilisent le même module et le même groupe de connexions à PostgreSQL. La fonction reçoit séparément le texte SQL et les valeurs. Une saisie utilisateur ne devient donc pas une instruction SQL. Les rares fragments assemblés par les modèles sont fixes ; les valeurs restent des paramètres.

Contrôle de propriété dans les requêtes SQL

Pour les ressources privées, les requêtes filtrent par l'identifiant de l'utilisateur authentifié. Les mouvements passent par leur actif pour retrouver ce propriétaire. Cette convention doit être appliquée et testée sur chaque route : aucun découpage ne dispense de vérifier les autorisations.

La table des mouvements ne porte pas de colonne de propriétaire, puisque c'est l'actif qui référence l'utilisateur : le cloisonnement passe donc par une jointure à l'intérieur de la même requête. Y compris à l'insertion, ce qui est le cas le plus intéressant.

`backend/src/models/transaction.js` : l'insertion filtrée par jointure.

```
const { rows } = await executer(
  `INSERT INTO transaction (actif_id, sens, quantite, prix_unitaire, frais, frais_montant, frais_unite,
date_transaction, note)
  SELECT a.id, $3, $4, $5, $6, $7, $8, $9, $10
  FROM actif a
  WHERE a.id = $1 AND a.utilisateur_id = $2
  RETURNING id, actif_id, sens, quantite, prix_unitaire, frais, frais_montant, frais_unite,
date_transaction, note`,
  [
    actifId,
    utilisateurId,
    sens,
    quantite,
    prixUnitaire,
    frais,
```

```

    // Les deux colonnes de frais sont posées ensemble : la contrainte de cohérence
    // refuse une contre-valeur en euros qui ne serait pas égale au montant prélevé
    // lorsque celui-ci est déjà en euros.
    fraisMontant ?? frais,
    fraisUnite ?? 'EUR',
    dateTransaction,
    note ?? null,
  ]
);

```

J'ai choisi `INSERT ... SELECT` pour réunir le **contrôle de propriété** et l'écriture dans une même requête. Avec un contrôle séparé suivi d'un `INSERT ... VALUES`, l'actif pourrait changer entre les deux opérations. Ici, aucune ligne n'est créée si l'actif n'appartient pas au demandeur, et la fonction renvoie `null`.

Un actif inexistant et un actif appartenant à un autre compte produisent tous deux un 404. Le code de filtrage est présenté ci-dessus. Les essais de cloisonnement appartiennent à la campagne datée présentée en 3.9.

Transaction SQL et verrouillage des écritures

La base ne peut pas vérifier seule la règle « on ne vend pas plus que ce que l'on détient », comme expliqué en 3.4. Un contrôle dans le code reste toutefois insuffisant si deux ventes arrivent en même temps : elles peuvent lire le même solde et se croire toutes deux valides. J'exécute donc la lecture et l'écriture dans une même transaction.

`backend/src/db/index.js` : lecture et écriture sur la même connexion, tout ou rien.

```

// Exécute une unité de travail sur une seule connexion du pool. Le callback reçoit une
// fonction query liée au client : toutes ses lectures et écritures voient donc le même
// état transactionnel. Le rollback est systématique dès qu'une règle métier ou
// PostgreSQL refuse l'opération.
async function executerDansTransaction(operation) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const resultat = await operation(client.query.bind(client));
    await client.query('COMMIT');
    return resultat;
  } catch (erreur) {
    await client.query('ROLLBACK');
    throw erreur;
  } finally {
    client.release();
  }
}

```

La transaction garantit que l'opération est entièrement validée ou entièrement annulée. Elle ne suffit pas à ordonner deux écritures simultanées. Le verrou posé sur la ligne de l'actif remplit ce second rôle.

`backend/src/models/actif.js` : la lecture verrouillante.

```

async function verrouillerParIdEtUtilisateur(id, utilisateurId, executer) {
  const { rows } = await executer(
    `SELECT ${CHAMPS}
     FROM actif
     WHERE id = $1 AND utilisateur_id = $2
     FOR UPDATE`,
    [id, utilisateurId]
  );
  return rows[0] || null;
}

```

`FOR UPDATE` verrouille la ligne de l'actif jusqu'à la fin de la transaction. Une autre opération sur la même position doit attendre, puis relire l'état validé. J'ai corrigé tardivement un point important : tous les mouvements prennent ce verrou, achats compris. Une vente en attente relit donc l'achat qui vient d'être

validé. Si elle passe avant lui et que la quantité est insuffisante, elle est refusée. Le verrou porte sur l'actif, car c'est la ressource commune aux mouvements concernés.

Le contrôle lui-même rejoue l'historique complet dans l'ordre du domaine :

backend/src/services/portefeuille.js : aucun préfixe chronologique ne peut passer sous zéro.

```
function verifierHistoriqueSortiesAutorisees(transactions) {
  let detenu = 0n;

  for (const transaction of trierChronologiquement(transactions)) {
    const quantite = versUnites(transaction.quantite, ECHELLE_QUANTITE);

    if (transaction.sens === 'achat') {
      detenu += quantite;
      continue;
    }

    if (quantite > detenu) {
      const erreur = new ErreurValidation(
        `Quantité insuffisante : vous détenez ${versChaine(detenu, ECHELLE_QUANTITE)} sur cet actif.`
      );
      erreur.mouvement = transaction;
      throw erreur;
    }

    detenu -= quantite;
  }
}
```

Les commentaires internes ont été retirés de cet extrait pour la lecture.

Vérifier le seul solde final aurait laissé passer une vente antidatée : on peut détenir une quantité suffisante aujourd'hui et ne pas l'avoir détenue au mois de juin. Le contrôle porte donc sur chaque étape, et il s'applique aussi au retrait d'un mouvement : retirer un achat ancien peut priver de sa contrepartie une vente enregistrée plus tard. Le raisonnement complet et sa preuve figurent en 3.6.

Cache des cours avec Redis

J'ai utilisé PostgreSQL pour les données relationnelles et Redis pour le cache. Ces deux usages me permettent de travailler les accès SQL et NoSQL sur des besoins du projet.

Le cache répond aux contraintes décrites en 1.4. Sans lui, consulter six positions demande six appels externes, puis six autres au rafraîchissement. Les quotas gratuits seraient rapidement consommés pendant une démonstration. J'ai retenu Redis plutôt qu'un cache dans le processus, car il gère la durée de vie de chaque clé, reste disponible après un redémarrage de l'API et peut être inspecté en ligne de commande.

Le service utilise deux familles de clés : `cours:v2:{TYPE}:{SYMBOLE}` pour le cours encore valable et `cours:v2:dernier-connu:{TYPE}:{SYMBOLE}` pour la dernière valeur reçue.

Au départ, la clé ne contenait pas la classe d'actif. Deux actifs de classes différentes portant le même symbole pouvaient alors partager un cours. J'ai ajouté la classe à la clé et un préfixe de version qui rend l'ancien format inaccessible.

La durée de vie dépend du rythme de cotation : 120 secondes pour une cryptomonnaie, 3 600 pour une devise, 600 pour un métal et 300 pour une action. Après chaque appel réussi, une seconde clé sans expiration conserve le dernier cours connu. Elle sert de valeur de secours lorsqu'un fournisseur ne répond pas.

Les clés actuelles incluent la classe d'actif, par exemple `cours:v2:crypto:BTC` et `cours:v2:dernier-connu:crypto:BTC`. La seconde n'a pas de délai d'expiration, mais elle reste liée à la durée de vie du stockage Redis ; elle n'est pas une sauvegarde permanente.

Le cache reste une optimisation. S'il est indisponible, le client journalise l'erreur une seule fois, tente de se reconnecter avec un délai progressif plafonné et abandonne l'opération après 200 millisecondes. Le service interroge alors directement le fournisseur : arrêter Redis ne bloque pas l'application.

3.6 Composants métier côté serveur

La logique serveur repose principalement sur le moteur de calcul, les adaptateurs de cours et la validation des entrées.

Le moteur de calcul et ses sept règles

J'ai retenu les règles suivantes pour rendre le suivi cohérent et pouvoir vérifier les résultats à la main.

#	Règle	Pourquoi
1	Le prix de revient intègre les frais d'achat	Le coût réel comprend ce qu'il a fallu payer pour acquérir le titre. L'exclure afficherait une plus-value flatteuse.
2	Un achat recalcule le prix de revient en moyenne pondérée	Convention de suivi retenue pour WalletWatch. Elle fournit un coût moyen par unité ; ce calcul n'est pas une déclaration fiscale.
3	Une vente ne modifie pas le prix de revient	Vendre ne change pas ce qu'ont coûté les titres restants. Seule la quantité détenue diminue.
4	La plus-value réalisée vaut quantité × (prix de vente - prix de revient) - frais de vente	C'est le gain effectivement encaissé sur les titres cédés, net des frais.
5	Une vente totale suivie d'un rachat repart d'un prix de revient neuf	La position précédente est soldée : le nouveau lot n'a aucun lien de coût avec l'ancien.
6	Les mouvements sont traités par ordre chronologique	Saisir après coup un mouvement ancien doit donner le même résultat que l'avoir saisi dans l'ordre.
7	Une sortie non marchande réduit la quantité et le coût au prorata	Un retrait ou un transfert ne dégage aucun produit. Réduire la quantité sans le coût ferait mécaniquement monter le prix de revient unitaire, ce qui serait faux.

Sans la règle 6, le résultat dépendrait de l'ordre de saisie. Le moteur trie donc les mouvements par date, puis par identifiant lorsque deux mouvements partagent la même date.

Le moteur utilise des fonctions pures : elles reçoivent des mouvements et un cours, puis renvoient un résultat. Elles n'accèdent ni à la base, ni au réseau, ni au cache. Je peux donc les tester isolément et reproduire leur calcul à la main.

backend/src/services/calculPortefeuille.js : le cœur du déroulé d'une position.

```
// Règles 1 et 2 : moyenne pondérée, frais d'achat inclus au coût de revient.
const coutExistant = multiplier(pru, ECHELLE_PRU, quantite, ECHELLE_QUANTITE, ECHELLE_PRU);
const coutAchat =
    multiplier(prixUnitaire, ECHELLE_PRIX, quantiteTransaction, ECHELLE_QUANTITE, ECHELLE_PRU) +
    frais;
const nouvelleQuantite = quantite + quantiteTransaction;

// Un achat de quantité nulle est refusé à la validation ; la garde évite
// malgré tout toute division par zéro sur une donnée inattendue.
pru =
    nouvelleQuantite === 0n
    ? 0n
    : diviser(
        coutExistant + coutAchat,
        ECHELLE_PRU,
        nouvelleQuantite,
        ECHELLE_QUANTITE,
        ECHELLE_PRU
    );
quantite = nouvelleQuantite;
```

Cet extrait correspond au traitement d'un achat, à l'intérieur de la branche achat. Le déroulé complet, avec les ventes et les sorties non marchandes, reste disponible dans le fichier cité.

Au départ, deux fonctions de calcul supposaient que leurs valeurs utilisaient la même échelle. Elles ne fonctionnaient que parce que deux constantes étaient égales. Modifier une seule constante aurait faussé le moteur d'un facteur dix sans produire d'erreur. J'ai d'abord ajouté un test qui rejoue le même calcul avec plusieurs échelles, puis j'ai passé chaque échelle en paramètre.

Une seule traversée produit l'état après chaque mouvement et l'état final. Calculer le prix de revient à une date donnée demande de rejouer tout l'historique dans le même ordre et avec les mêmes arrondis. Centraliser ce traitement évite d'entretenir deux versions de la règle.

Des calculs métier en entiers à échelle fixe

Le moteur ne calcule aucun montant en virgule flottante. Chaque nombre est représenté par un entier `BigInt` associé à une échelle fixe ; à l'échelle 2, la valeur 1,5 devient l'entier 150. En virgule flottante, $0.1 + 0.2$ vaut 0.30000000000000004 , ce qui ne convient pas à des quantités pouvant atteindre dix-huit décimales.

Les fonctions `versUnites` et `versChaine` convertissent les chaînes décimales en entiers de calcul, puis effectuent l'opération inverse. Transformer en chaîne un nombre déjà arrondi ne permet pas de retrouver les chiffres perdus. L'annexe D.1 complète ce point avec la règle d'arrondi des divisions.

Le moteur utilise 18 décimales pour les quantités et les cours, 24 pour le prix de revient et 2 pour les montants en euros. La précision des quantités permet notamment de représenter de petites fractions d'ETH. La précision supérieure du prix de revient limite les arrondis intermédiaires ; les résultats en euros restent arrondis selon la règle du projet.

Ces échelles ont été relevées en cours de projet, et la correction n'est pas rétroactive : un cours arrondi à zéro perdait sa valeur d'origine ; augmenter la précision ne la reconstitue pas.

L'arrondi retenu est au plus proche, avec les demis éloignés de zéro. Il traite de la même façon les gains et les pertes. Pour la répartition du patrimoine, arrondir chaque part séparément peut produire un total de 99,98 % ou 100,01 %. La dernière part reçoit donc le reliquat afin d'afficher exactement 100 %.

Les adaptateurs de cours et le contrôle des réponses

Les adaptateurs des quatre classes d'actifs exposent la même signature : `getCours(symbole) → { symbole, cours_eur, horodatage }`. Le service métier utilise ainsi le même format, quel que soit le fournisseur interrogé.

Le service de cours suit le déroulé ci-dessous. La validation du symbole dans l'adaptateur dépend de la classe : les actions utilisent une liste fermée, tandis que les suggestions crypto et devise ne sont pas des listes d'interdiction.

Étape	Résultat
Chercher un cours frais par classe et symbole	Retour du cache s'il existe
Appeler l'adaptateur correspondant	Validation de la réponse et conversion éventuelle en euros
Si l'appel réussit	Mise à jour du cours frais et du dernier cours connu
Si l'appel échoue	Recherche du dernier cours connu pour la même classe et le même symbole
Sans dernier cours connu	Erreur de cotation signalée ; aucune valeur nulle inventée

L'adaptateur des actions enchaîne trois fournisseurs et convertit le résultat avec le taux euro-dollar déjà en cache.

backend/src/adaptateurs/actions.js : le passage d'un fournisseur au suivant.

```
const echecs = [];  
let cotation = null;  
  
for (const fournisseur of disponibles) {  
  try {  
    cotation = await fournisseur.interroger(symboleNormalise, fournisseur.cle);  
    break;  
  } catch {  
    echecs.push(fournisseur.nom);  
  }  
}
```

Deux fournisseurs signalent certaines erreurs dans le contenu de leur réponse plutôt que par le code HTTP (voir 4.1). La fonction commune `validerPrix` contrôle donc aussi la valeur reçue. Si le prix est nul, absent ou non numérique, le service essaie le fournisseur suivant.

La validation des entrées

Une fonction vérifie les données avant le contrôleur à l'aide de schémas Zod. Le contrôleur reçoit ainsi un corps déjà contrôlé et normalisé.

backend/src/validation/transaction.js : la validation des montants, sans conversion en nombre.

```
function nombreDecimal({ decimalesMax, chiffresEntiersMax, strictementPositif }) {  
  const motif = new RegExp(`^\\d{1,${chiffresEntiersMax}}(\\.\\d{1,${decimalesMax}})?$`);  
  
  return z  
    .union([z.string(), z.number()])  
    .transform((valeur) => versNotationPositionnelle(String(valeur).trim()))  
    .refine((valeur) => motif.test(valeur), {  
      message:  
        `La valeur doit être positive, comporter au plus ${decimalesMax} décimales ` +  
        `et ${chiffresEntiersMax} chiffres avant la virgule.`  
    })  
    // Strictement positif se lit sur les chiffres, sans conversion : passer par Number  
    // ferait rentrer le flottant que tout le reste de la chaîne tient à l'écart.  
    .refine((valeur) => !strictementPositif || /[1-9]/.test(valeur), {  
      message: 'La valeur doit être strictement positive.'  
    })  
  };
```

Le validateur accepte aussi des nombres pour compatibilité. Pour conserver toutes les décimales, le client doit transmettre une chaîne : `String()` ne répare pas une perte de précision déjà survenue. La valeur normalisée est ensuite contrôlée et transmise en chaîne jusqu'à la base.

Deux contrôles complètent ce schéma. L'option `.strict()` **refuse** tout champ non attendu : une inscription ne peut donc pas ajouter un rôle d'administrateur dans le corps de la requête. De plus, `actif_id` vient de l'adresse et jamais du corps envoyé.

La règle de vente, et son symétrique au retrait

La règle « on ne vend pas plus que ce que l'on détient » est appliquée par le service des mouvements, dans la transaction verrouillée décrite en 3.5. Elle porte sur chaque étape de l'historique, et non sur le seul solde final.

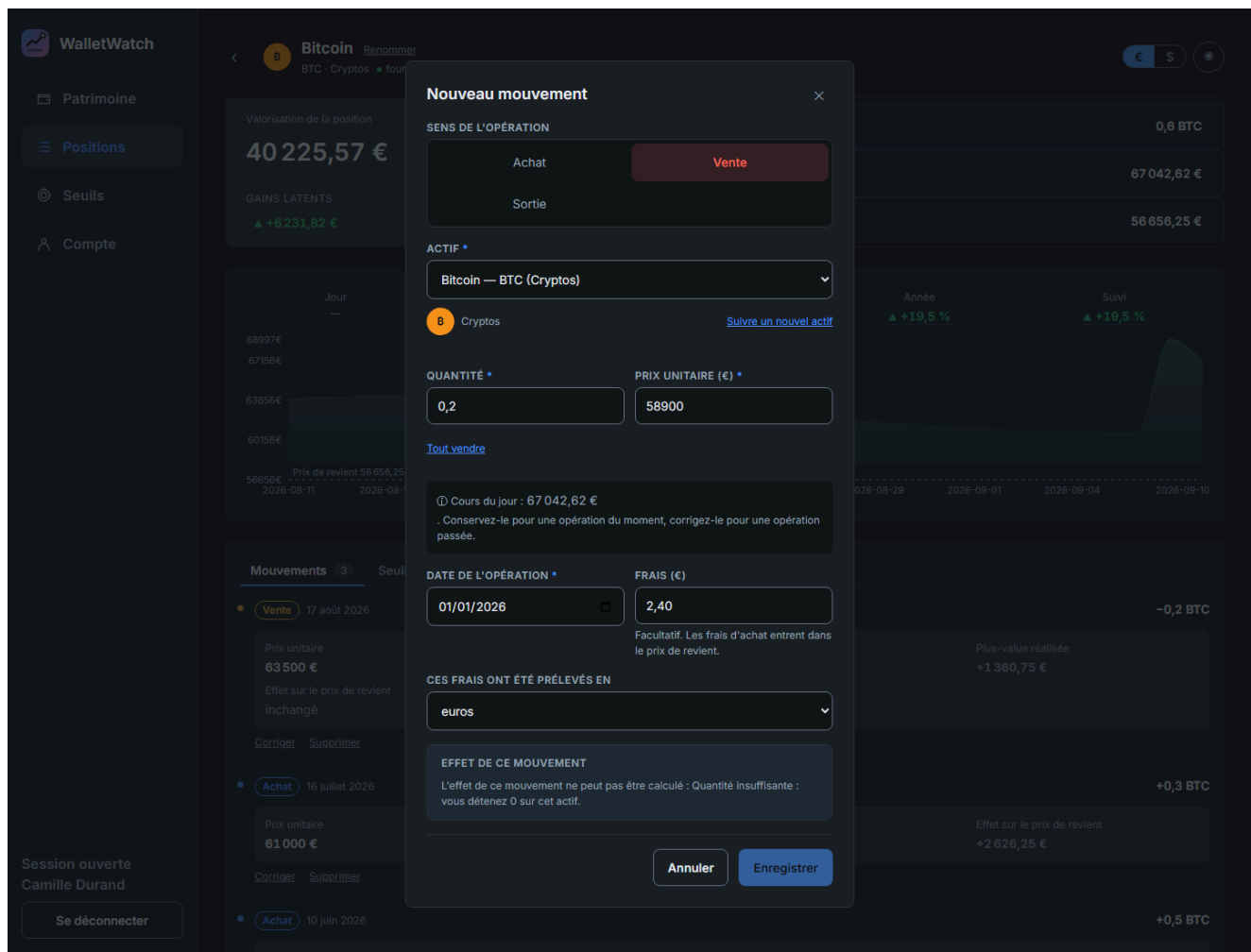


Figure 15 : Simulation d'une vente avant le premier achat : la quantité disponible à cette date est nulle et le serveur refuse le calcul. Capture du 10 septembre 2026.

La capture montre une vente datée avant le premier achat de la position. Le solde actuel serait suffisant, mais la position était vide à la date demandée. La simulation refuse donc l'opération avant tout enregistrement.

Le même contrôle s'applique à la suppression d'un mouvement. Le service rejoue l'historique sans celui-ci et refuse la suppression si une vente ultérieure devient impossible. Le fichier complet reste dans le dépôt ; aucune suppression n'a été exécutée pendant les captures du 10 septembre.

`backend/src/services/transaction.js` : le même invariant, avec un message adapté à la suppression.

```
// L'invariant reste exactement celui de la création : aucun préfixe chronologique
// ne peut passer sous zéro. Seul le message change.
try {
```

```
verifierHistoriqueSortiesAutorisees(historiqueApresSuppression);  
} catch (erreur) {  
    if (erreur instanceof ErreurValidation) throw new ErreurValidation(MESSAGE_RETRAIT_IMPOSSIBLE);  
    throw erreur;  
}
```

Les tests d'intégration avec PostgreSQL vérifient notamment les accès concurrents et la conservation d'une quantité valide. Ils sont distincts de la simulation montrée ici, qui ne réalise aucune écriture.

L'évaluation des seuils

L'évaluation des seuils se déclenche à chaque consultation du tableau de bord. Le service compare les seuils actifs au cours de l'actif ou à la valeur totale du patrimoine, puis marque ceux qui sont franchis. L'application ne fait aucun envoi : elle constate le franchissement lorsque l'utilisateur consulte l'écran. Une vérification en arrière-plan aurait demandé un ordonnanceur, un canal d'envoi et une gestion des doublons, hors du périmètre retenu.

Pour chaque seuil non franchi, le service calcule aussi l'écart restant en pourcentage. La lecture des valeurs et la mise à jour des statuts restent séparées ; une simple consultation ne doit pas écrire dans la base.

3.7 Éléments de sécurité de l'application

WalletWatch manipule la valeur et la répartition d'un patrimoine. Aucun mouvement de fonds n'y transite, mais ces informations restent sensibles. Les contrôles sont répartis entre l'interface, l'API et les requêtes SQL. Cette section présente le risque couvert par chaque mesure ainsi que les limites du projet.

Mots de passe et accès au compte

Les mots de passe sont hachés avec bcrypt, avec un coût de 10. Les profils renvoyés par l'API ne contiennent ni le mot de passe ni son empreinte. Les traitements qui vérifient un mot de passe accèdent séparément à cette colonne.

La connexion renvoie le même message pour une adresse inconnue ou un mot de passe incorrect. Une comparaison bcrypt factice est aussi réalisée lorsque le compte n'existe pas, pour réduire une différence évidente de temps de réponse. Ce dispositif ne constitue pas une preuve de temps constant. L'inscription peut encore signaler qu'une adresse est déjà utilisée : l'existence d'un compte n'est donc pas entièrement confidentielle.

Le changement de mot de passe et la suppression du compte sont contrôlés côté serveur. Demander un mot de passe dans le seul formulaire ne suffirait pas, puisque l'API peut être appelée directement.

Jeton et propriété des données

Le jeton est envoyé dans l'en-tête Authorization. Avant d'autoriser l'accès, le serveur impose l'algorithme HS256, vérifie la signature et l'expiration, puis relit le compte dans la base. Un compte absent ou désactivé ne peut plus utiliser une route privée. Le traitement complet reste disponible dans `backend/src/middlewares/authentifieur.js`.

Une borne `jetons_invalides_avant` permet de refuser les jetons antérieurs à un changement de mot de passe. Le code compare à la seconde : un jeton émis dans la même seconde que la borne reste accepté. Je ne présente donc pas cette révocation comme instantanée sans exception.

Le contrôle de propriété complète l'authentification. Dans les modèles, l'identifiant de l'utilisateur connecté fait partie de la requête SQL. Une ressource appartenant à un autre compte produit un 404, comme une ressource absente. Connaître son identifiant ne suffit donc pas pour y accéder. Le contrôle des rôles existe dans le code, mais l'espace d'administration n'est pas livré.

Entrées, SQL et affichage

Les schémas Zod contrôlent les corps et paramètres avant le traitement. Le mode strict refuse les champs inconnus, par exemple un champ `role` ajouté à une inscription. Le serveur ne prend pas l'identifiant du propriétaire dans le corps de la requête.

Les valeurs entrent dans le SQL sous forme de paramètres, séparés du texte de la requête. Certains fragments SQL fixes peuvent être choisis par le programme ; aucune saisie utilisateur ne doit devenir un fragment de requête. Le rôle applicatif a également des droits limités sur la base.

React échappe les valeurs insérées comme texte. Le projet n'utilise pas `dangerouslySetInnerHTML`. Cette protection doit être conservée pour le pseudonyme, les noms d'actifs et les autres données affichées. Elle ne dispense pas de valider les longueurs et les formats attendus.

Limitation des tentatives et récupération

Les routes de connexion et de récupération limitent les tentatives par adresse électronique sur une fenêtre de quinze minutes. Cette protection réduit les essais répétés sur un compte. Elle conserve une limite : un tiers peut épuiser le quota d'une adresse et gêner temporairement son titulaire. Une exploitation publique demanderait de revoir aussi l'identification fiable de l'adresse cliente derrière le relais.

La récupération produit une clé aléatoire de 32 octets, valable une heure et à usage unique. La base ne conserve que son empreinte SHA-256. Par défaut, la réponse ne livre pas cette clé. Un réglage de démonstration peut l'afficher ; il permet alors à une personne connaissant l'adresse d'un compte de demander sa réinitialisation. **Ce mode est réservé aux comptes fictifs d'une instance isolée.** L'envoi sécurisé au titulaire du compte n'est pas implémenté.

Navigateur, origines et secrets

La configuration CORS encadre la lecture des réponses par un navigateur. Elle ne remplace pas l'authentification et ne bloque pas un client HTTP direct. L'authentification utilise un en-tête ajouté par l'interface, et non un cookie joint automatiquement : cela réduit le risque de CSRF classique. Si ce stockage évoluait vers un cookie, la protection devrait être réexaminée.

Le jeton reste dans `sessionStorage`, accessible au JavaScript de la page. Une faille XSS pourrait donc le voler. Nginx sert une politique de sécurité du contenu, notamment `script-src 'self'`, qui limite les scripts autorisés. Cette politique réduit certains risques sans garantir l'absence de faille.

Le secret JWT et les clés des fournisseurs doivent rester dans la configuration non publiée. Les identifiants fictifs présents dans les jeux de démonstration ne doivent jamais servir à de vrais comptes. La pile locale ne fournit pas HTTPS par défaut ; une ouverture publique demande de chiffrer le transport.

Contrôles et limites

La campagne de vérification conservée du 7 septembre porte notamment sur les routes sans jeton, les jetons invalides, les champs interdits, les identifiants de ressource mal formés et le cloisonnement entre comptes. Les résultats attendus sont un 401 pour une session invalide, un 400 pour une entrée rejetée et un 404 pour une ressource d'autrui. La section 3.9 distingue cette campagne des essais ciblés du 10 septembre.

Ces contrôles ne sont pas un audit de sécurité exhaustif. Le mode de récupération de démonstration, le transport HTTP, le stockage du jeton et les limites de protection contre les tentatives répétées restent des points à traiter avant un usage réel.

3.8 Jeu d'essai de la fonctionnalité la plus représentative

La fonctionnalité représentative est le calcul du prix de revient et des plus-values. Elle relie la saisie de mouvements, l'accès aux données, les traitements du serveur et leur restitution dans l'interface.

Le premier jeu d'essai utilise des nombres simples pour vérifier le moteur. Le second reprend une simulation dans l'interface locale et compare les valeurs affichées au calcul attendu.

Premier jeu d'essai : le scénario de référence

Données en entrée. Un utilisateur suit un actif. Il réalise trois opérations, puis consulte sa position alors que le cours vaut 130,00 €.

Date	Opération	Quantité	Prix unitaire	Frais
10/01/2026	achat	10	100,00 €	5,00 €
10/02/2026	achat	10	120,00 €	5,00 €
10/03/2026	vente	5	150,00 €	5,00 €

Données attendues. Elles sont calculées à la main, à partir des règles énoncées en 3.6, avant toute exécution.

Le premier achat porte sur 10 titres à 100,00 €, soit 1 000,00 €, auxquels s'ajoutent 5,00 € de frais : la position coûte 1 005,00 € pour 10 titres, donc **100,50 € l'unité** (règle 1). Le second achat porte sur 10 titres à 120,00 €, plus 5,00 € de frais, soit 1 205,00 €. Le nouveau prix de revient est le coût total divisé par la quantité totale (règle 2) :

```
prix de revient = (100,50 × 10 + 120,00 × 10 + 5,00) / (10 + 10)
                = (1 005,00 + 1 205,00) / 20
                = 2 210,00 / 20
                = 110,50 €
```

Le résultat n'est pas la moyenne arithmétique des deux prix d'achat, qui vaudrait 110,00 €. Les 10,00 € de frais ajoutent 0,50 € par titre, ce qui porte le prix de revient à 110,50 €. Ignorer ces frais surestimerait la plus-value.

La vente du 10/03 porte sur 5 titres à 150,00 €, avec 5,00 € de frais. Le prix de revient **reste à 110,50 €** (règle 3) : vendre ne change pas ce qu'ont coûté les titres restants. Seule la quantité détenue passe de 20 à 15. La plus-value réalisée se calcule sur les seuls titres cédés (règle 4) :

```
plus-value réalisée = 5 × (150,00 - 110,50) - 5,00
                   = 5 × 39,50 - 5,00
                   = 197,50 - 5,00
                   = 192,50 €
```

Au cours de 130,00 €, la position vaut alors 15 × 130,00 = 1 950,00 €, pour un coût de 15 × 110,50 = 1 657,50 €, soit une plus-value latente de 292,50 €, ou +17,65 %.

Grandeur	Valeur attendue
Quantité détenue	15
Prix de revient unitaire	110,50 €
Coût total de la position	1 657,50 €
Valeur au cours de 130,00 €	1 950,00 €
Plus-value latente	292,50 € (+17,65 %)
Plus-value réalisée	192,50 €

Données obtenues. Le scénario a été rejoué directement avec les fonctions du moteur le 10 septembre 2026, sans base ni réseau. Les six grandeurs attendues sont retrouvées. Il figure aussi dans la suite de tests, qui compare des chaînes de caractères et non des nombres à virgule flottante : comparer 110.5 à 110.5 est exact, comparer deux flottants à une tolérance près ne prouverait rien sur un calcul monétaire.

```
// Règle 2, cas vérifiable de tête : (1005 + 1205) / 20 = 110,50.
expect(position.pru).toBe('110.5');

// 5 × (150 - 110,50) - 5 = 192,50.
expect(position.plus_value_realisee).toBe('192.50');
```

Analyse des écarts. Aucun. Les valeurs obtenues sont identiques aux valeurs attendues, au centime près, sur les six grandeurs.

Trois cas limites du même fichier de tests portent sur l'exactitude plutôt que sur la règle : l'addition de 0,1 et 0,2 rend exactement 0,3 ; une quantité de 0,00000001 est représentée sans perte ; et l'inversion de l'ordre de saisie de deux mouvements donne rigoureusement le même résultat, ce qui vérifie la règle 6.

Second jeu d'essai : simulation sur l'application réelle

Le 10 septembre 2026, j'ai repris le scénario dans l'instance locale de démonstration, avec le compte fictif Camille Durand. La simulation ne crée aucun mouvement.

Entrées. La position Bitcoin contient un achat de 0,5 BTC à 54 000 € avec 15 € de frais, un achat de 0,3 BTC à 61 000 € avec 10 € de frais et une vente de 0,2 BTC à 63 500 € avec 8 € de frais. Les dates visibles sont respectivement le 10 juin, le 16 juillet et le 17 août 2026. Je simule un achat au 10 septembre de 0,2 BTC à 58 900 €, avec 2,40 € de frais.

Calcul attendu. Les deux premiers achats coûtent $27\,015 + 18\,310 = 45\,325$ € pour 0,8 BTC, soit un prix de revient de 56 656,25 €. La vente laisse 0,6 BTC avec le même prix de revient. Elle produit $0,2 \times (63\,500 - 56\,656,25) - 8 = 1\,360,75$ € de plus-value réalisée.

L'achat simulé coûte $0,2 \times 58\,900 + 2,40 = 11\,782,40$ €. Le prix de revient attendu devient :

```
(0,6 × 56 656,25 + 11 782,40) / 0,8 = 57 220,1875 €
Écart de prix de revient : 57 220,1875 - 56 656,25 = 563,9375 €
```

Grandeur	Attendu à l'affichage	Obtenu le 10 septembre
Prix de revient avant simulation	56 656,25 €	56 656,25 €
Plus-value de la vente existante	+1 360,75 €	+1 360,75 €
Montant de l'achat simulé, hors frais	11 780 €	11 780 €
Frais	2,40 €	2,4 €
Quantité après simulation	0,8 BTC	0,8 BTC
Nouveau prix de revient	57 220,19 €	57 220,19 €
Écart	+563,94 €	+563,94 €

La [figure 10](#) montre le récapitulatif et la [figure 12](#) la position avant enregistrement. Les valeurs affichées correspondent au calcul attendu. L'affichage retire le zéro final de 2,40 €, sans changer sa valeur. Le prix de revient et son écart sont arrondis au centime pour la lecture ; le moteur conserve une précision interne supérieure.

Analyse. Aucun écart de calcul n'a été observé sur ce scénario. Ce résultat vérifie la liaison entre le formulaire, la simulation serveur et le rendu. Il ne teste pas une écriture en base ni une vente concurrente : ces contrôles relèvent des tests d'intégration présentés en 3.9.

3.9 Tests et qualité du code

Une stratégie dictée par l'architecture

La stratégie de tests découle directement du découpage en couches décrit en 2.2, et c'est sa principale justification pratique : le métier est testable sans base et sans HTTP.

Le moteur de calcul utilise des fonctions pures qui reçoivent des mouvements et un cours. Les services d'orchestration reçoivent leurs modèles en paramètre : les tests peuvent fournir un modèle de substitution sans modifier le code du service.

Ce second point n'était pas prévu à la conception : il est né d'un échec. Les premiers tests du service de portefeuille remplaçaient les fonctions des modèles par des espions, sans effet, parce que le service détenait sa propre référence obtenue à l'importation, et les tests tentaient réellement de joindre PostgreSQL. Plutôt que de contourner par une mécanique de simulation plus fine, j'ai rendu les modèles injectables.

Les chiffres

Les trois suites s'exécutent avec le même lanceur, sous des configurations adaptées au serveur, à l'intégration PostgreSQL et à l'interface. Les deux suites qui ne demandent ni base ni réseau ont été rejouées le 10 septembre 2026 sur le commit de référence a8a5e6d : elles rendent les mêmes totaux qu'à la campagne du 7 septembre. La suite d'intégration, qui exige un PostgreSQL dédié, compte toujours trois fichiers et vingt-deux tests au commit de référence, mais son résultat reste celui du 7 septembre.

Périmètre	Commande	Fichiers	Tests	Dernière exécution
Serveur, sans base	npm test depuis backend/	32	410	10 septembre 2026
Serveur, contre PostgreSQL	npm run test:integration	3	22	7 septembre 2026
Interface	npm test depuis frontend/	23	359	10 septembre 2026

Le détail par fichier montre que les tests se concentrent sur les règles métier : 38 pour le moteur de calcul, 28 pour l'évaluation des seuils, 28 pour le portefeuille consolidé et 17 pour la validation des mouvements. Côté interface, 54 tests couvrent le formatage. Ces nombres ont été relevés lors de la réexécution du 10 septembre.

La troisième suite est apparue tard, pour une raison précise : un test à modèles simulés vérifie qu'un service **demande** un verrou, jamais que ce verrou **bloque**. Elle s'exécute donc contre un vrai PostgreSQL, dans un environnement conteneurisé dont le nom de projet, le volume et les ports sont distincts de ceux du développement.

Ce qui est couvert, et pourquoi

Le moteur de calcul fait l'objet de tests dédiés. Chaque règle a son test, avec des nombres choisis pour être vérifiables de tête, plus une série de cas limites : petite quantité de 0,00000001, puis cas à dix-huit décimales, position soldée puis rachetée, saisie rétroactive, mouvements de même date départagés par leur identifiant, et le contrôle d'exactitude sur $0,1 + 0,2$.

La validation des entrées est testée sur ses refus autant que sur ses acceptations. Les tests vérifient notamment le rejet d'une date future, d'une quantité négative, d'une décimale de trop et d'un champ inattendu.

Les adaptateurs de cours sont testés avec des réponses simulées qui reproduisent aussi les erreurs observées : réponse HTTP 200 contenant un message d'erreur, cours nul pour un symbole inconnu ou clé absente. Ces tests vérifient le passage au fournisseur suivant, puis au dernier cours connu. **Les contrôles placés avant les routes** sont testés séparément. **Côté interface**, les tests portent sur ce que voit ou fait l'utilisateur : formatage des valeurs, protection des pages privées, formulaires de saisie et états de chargement, d'erreur ou de succès des sept écrans.

Limites des vérifications

Il n'y a **pas de tests de bout en bout automatisés**. Piloter un navigateur avec une base et un jeu de données dédiés aurait demandé plusieurs jours sur les six semaines du projet. J'ai préféré vérifier manuellement chaque lot avec une vraie base et un vrai navigateur, puis conserver les résultats. C'est un choix lié au calendrier ; sur un projet appelé à durer, j'automatiserais ces parcours en priorité.

La suite d'intégration vérifie ce que les tests unitaires ne peuvent pas reproduire seuls : deux opérations réellement concurrentes, l'absence d'écriture pendant une lecture et l'accès aux routes avec PostgreSQL. Pour contrôler l'absence d'écriture, le test compte les lignes avant et après l'appel.

Il n'y a **pas d'appel réel aux fournisseurs de cours** dans la suite automatisée. Un tel test dépendrait de leur disponibilité et rendrait le résultat instable. Leurs réponses sont donc simulées, puis les appels réels sont vérifiés à la main. Je ne publie pas non plus de **mesure de couverture** : un pourcentage de lignes exécutées ne suffit pas à juger la qualité des vérifications.

Deux tests qui ont réellement servi

Deux tests méritent d'être détaillés, car ils ont permis de détecter ou de prévenir un défaut concret.

Le premier vérifie l'exactitude décimale :

```
// L'arithmétique entière doit rester exacte là où le flottant dérive.
it('reste exact sur des quantités à 8 décimales', () => {
  const position = calculerPosition([
    achat('0.1', '100.00', '0', '2026-01-10', 1),
    achat('0.2', '100.00', '0', '2026-02-10', 2),
  ]);
  expect(position.quantite_detenue).toBe('0.3');
  expect(position.pru).toBe('100');
});
```

Il a échoué le jour où une conversion en nombre s'était glissée dans le chemin de calcul, et c'est ainsi que le défaut décrit en 4.1 a été trouvé.

Le second couvre la vente rétroactive. Je l'ai ajouté après avoir découvert le défaut, avec les dix-sept cas du service des mouvements. Il vérifie maintenant que cette régression ne revient pas.

Qualité du code

Le code est analysé statiquement par `oxlint`, qui ne signalait rien le 10 septembre dans les deux parties du dépôt. Il avait auparavant relevé cinq déclarations inutilisées côté serveur ; je les ai retirées sans reformater le reste. Les audits des dépendances sont datés et expliqués en 3.10. Les éléments préparatoires de l'administration restent une dette identifiée.

Les variables, fonctions, fichiers, tables, routes, commits et commentaires utilisent une convention française. Ce choix limite les variations de vocabulaire pour une même notion. Les identifiants historiques liés au nom CapitAll sont conservés lorsqu'un renommage aurait modifié les preuves ou demandé une migration.

3.10 Veille sur les vulnérabilités de sécurité

Ma méthode

J'ai utilisé les audits npm, les avis de sécurité des dépendances et leurs notes de version. Lorsqu'un avis apparaissait, je devais vérifier le paquet concerné, la version installée et l'usage de ce paquet dans l'application. Une correction proposée par un outil demande aussi de vérifier qu'elle reste compatible avec le projet.

Je conserve les résultats avec leur date. Un audit ne prouve pas qu'une application est sûre : il recherche les vulnérabilités connues des dépendances qu'il analyse. Il ne contrôle pas les autorisations SQL, les règles métier ou toute la configuration du serveur.

Un exemple rencontré avec le routeur

La comparaison de versions du routeur pendant le développement m'a conduit à lire l'avis GHSA-qwww-vcr4-c8h2. Il concerne un contournement de protection CSRF dans les API expérimentales RSC : une action peut être exécutée avant le rejet de la requête.

Au contrôle du 10 septembre 2026, l'avis donne 7.18.2 comme version corrigée, ce qui correspond au verrouillage actuel. L'application utilise le routage côté navigateur et n'emploie pas ces API RSC. J'en déduis que le chemin décrit par cet avis n'est pas utilisé ici ; cela ne dispense pas de maintenir le routeur à jour ni de vérifier d'autres avis.

Source : [avis officiel React Router GHSA-qwww-vcr4-c8h2](#), consulté le 10 septembre 2026.

J'ai daté cette vérification, car l'avis et la version installée peuvent évoluer. Une ancienne capture de l'audit ne décrit donc pas nécessairement l'état du 10 septembre.

L'audit des dépendances au jour de la remise

Le 10 septembre 2026, `npm audit` relève deux vulnérabilités de niveau modéré de chaque côté du dépôt. Elles viennent du même avis : une lecture de fichier arbitraire par traversée de chemin dans `@vitest/mocker`, corrigée en 4.1.11. Les fichiers de verrouillage installent 4.1.10.

Périmètre	Niveau	Paquets concernés	Version installée	Version corrigée
Serveur	modéré	<code>vitest</code> , <code>@vitest/mocker</code>	4.1.10	4.1.11
Interface	modéré	<code>vitest</code> , <code>@vitest/mocker</code>	4.1.10	4.1.11

Le paquet est une dépendance de développement dans les deux fichiers `package.json`. L'image de l'API utilise `npm ci --omit=dev`, tandis que celle de l'interface ne conserve que les fichiers compilés servis par Nginx : aucune n'embarque `vitest`. Le chemin décrit par l'avis passe par le serveur de test, lancé uniquement sur le poste de développement.

Je ne présente donc pas ces deux lignes comme une faille de l'application livrée, mais comme un point de veille daté. La montée en 4.1.11 reste à faire, avec une réexécution des suites ; elle ne relève pas d'une correction urgente en production.

Source : [avis GHSA-82fw-gwwq-j7x9](#), relevé par `npm audit` le 10 septembre 2026.

Ce que je retiens

Avant une mise à jour, je vérifie la correction annoncée et les éventuelles ruptures, puis je relance les tests et la construction. Le fichier de verrouillage conserve les versions utilisées. Je dois poursuivre cette veille après la remise : un avis peut être publié sans modification de mon code.

Cet audit ne remplace pas les contrôles propres au projet : filtrage des ressources par utilisateur, requêtes paramétrées, validation et configuration du déploiement. Ils sont présentés en 3.7.

3.11 Déploiement de l'application

Composition de la pile

La procédure de déploiement est fournie dans `docs/déploiement.md`. Le fichier `docker-compose.production.yml` assemble quatre services : PostgreSQL, Redis, l'API et Nginx pour l'interface compilée.

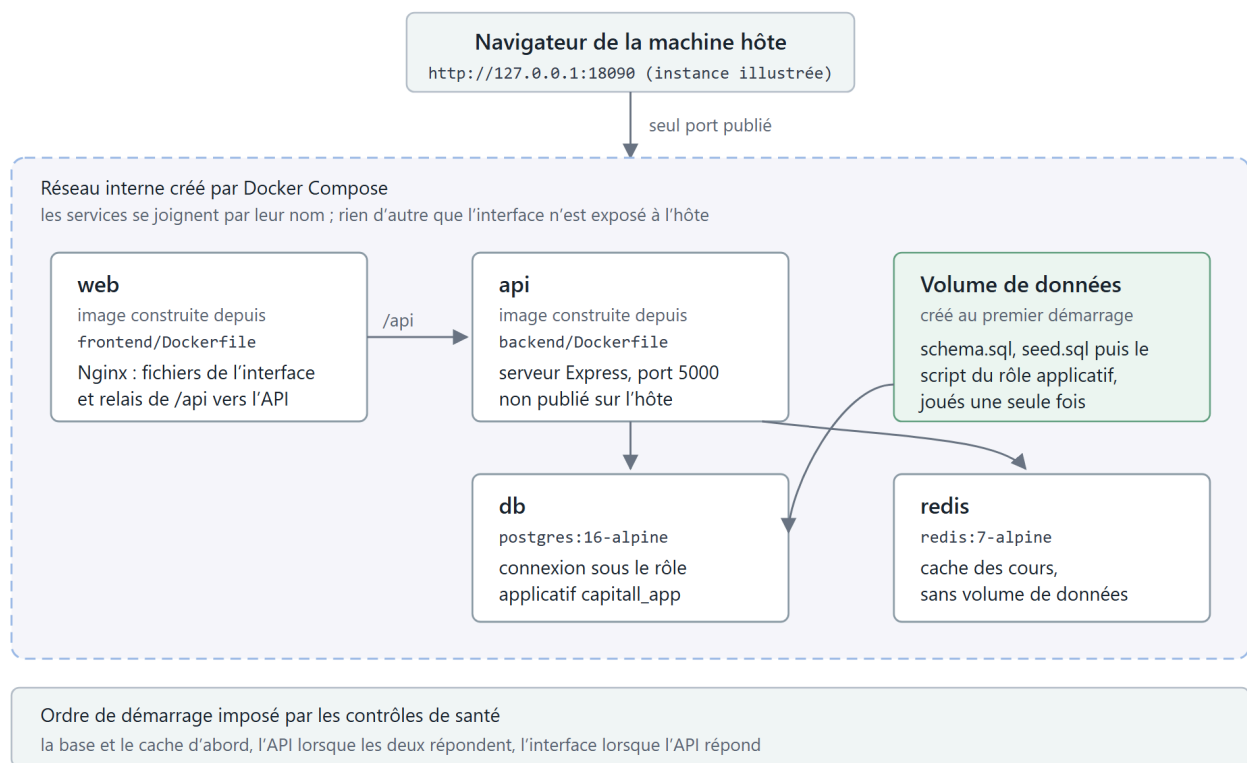


Figure 16 : Pile conteneurisée. Un seul port est publié sur la machine hôte ; la base, le cache et l'API ne sont joignables que depuis le réseau interne.

Dans cette pile, seul le port de Nginx est publié. Les requêtes `/api/...` sont relayées vers l'API sur le réseau interne, ce qui donne au navigateur une même origine pour l'interface et les données. Cette composition reste une base de démonstration locale ; ses limites pour une exploitation publique sont précisées plus bas.

Les images de l'API et de l'interface sont construites à partir du dépôt. L'interface est compilée avec Vite, puis ses fichiers sont servis par Nginx. L'API s'exécute sous l'utilisateur `node`. Les processus de travail Nginx utilisent un compte dédié, tandis que son processus maître démarre en `root`.

Configuration et premier démarrage

Le modèle `.env.example` indique les variables attendues. Les mots de passe, le secret JWT et les clés des fournisseurs doivent être renseignés hors du dépôt. L'API vérifie les variables nécessaires au démarrage, notamment la longueur minimale du secret JWT.

La commande documentée pour construire et lancer la pile est :

```
docker compose -f docker-compose.production.yml up -d --build
```

Elle suppose que la configuration a été préparée et que le port choisi est disponible. Sur une base neuve, les scripts d'initialisation créent le schéma, les données de démonstration et les droits nécessaires. Ils ne sont pas rejoués automatiquement sur un volume déjà initialisé.

Les données de démonstration et leurs comptes publics conviennent à une instance isolée d'examen. Il faut les retirer du processus d'initialisation avant tout usage avec de vrais comptes. La présence du mode de récupération affichant une clé impose la même restriction.

Vérifications et exploitation

Les contrôles de santé suivent le démarrage des services. `GET /api/sante` confirme que l'API répond à travers le relais, sans tester directement PostgreSQL ni Redis. J'ai complété ce contrôle par une connexion, une lecture des données et des vérifications propres à chaque service.

Les captures du 10 septembre proviennent de la pile locale accessible sur le port 18090. La connexion, les positions et la simulation y ont été observées. Cette observation ne constitue pas un nouveau test de redémarrage, de restauration ou de montée en charge.

PostgreSQL conserve ses données dans un volume. Recréer un conteneur ne remplace pas une migration du schéma. Le projet fournit `backend/db/migrer.js`, qui tient un registre des migrations et permet d'en consulter l'état. Avant d'appliquer une migration, il faut identifier la base visée et disposer d'une sauvegarde vérifiée. La restauration doit être essayée sur une base distincte, sans écraser la base de travail.

Redis n'a pas de volume persistant dans cette composition. Recréer son conteneur fait perdre le cache, y compris les derniers cours connus. L'application doit alors retrouver les fournisseurs ou signaler l'absence de cours.

Limites de la livraison locale

La pile présentée n'apporte pas HTTPS par défaut. Une ouverture à de vrais utilisateurs demande notamment un certificat, un canal sécurisé de récupération de compte, une gestion adaptée des secrets, des sauvegardes restaurables et un suivi des erreurs.

La documentation décrit aussi les pannes courantes : port occupé, rôle de base différent de celui du volume existant ou initialisation interrompue. Lors du diagnostic, je vérifie la configuration reçue par le conteneur réellement exécuté : modifier un fichier d'environnement ne met pas à jour un service déjà lancé.

Partie 4 : Bilan du projet

4.1 Difficultés rencontrées et solutions apportées

Protéger l'historique des mouvements

Un premier contrôle de vente ne vérifiait que la quantité disponible à la fin de l'historique. Il ne suffisait pas pour une vente antidatée. Deux requêtes simultanées pouvaient aussi lire le même solde avant d'écrire.

La solution consiste à reprendre les mouvements par date, puis par identifiant, et à vérifier que la quantité ne devient jamais négative. La lecture et l'écriture se font dans la même transaction SQL, sous verrou sur l'actif. La suppression d'un achat et la correction d'un mouvement suivent le même principe, car elles peuvent rendre une vente ultérieure impossible. Les essais avec une vraie base complètent les tests du moteur.

Gérer une panne de cache

Le cache devait rester facultatif, mais un essai avec Redis arrêté a révélé une attente trop longue. La limite de temps couvrait l'opération demandée, pas la connexion. Le client actuel interrompt aussi cette attente et laisse le service essayer le fournisseur. Cette erreur m'a montré qu'il fallait provoquer une panne réelle pour vérifier le comportement de secours.

Distinguer une date d'un instant

Une date de relevé passait par un objet `Date` JavaScript avant d'être envoyée au navigateur. La conversion de fuseau pouvait la faire apparaître la veille. Les dates calendaires sont maintenant rendues en chaînes AAAA-MM-JJ. Un jour de relevé et un horodatage de transaction ne représentent pas la même information.

Préserver les décimales

Une inversion de taux utilisait `Number`, alors que les autres calculs passaient par les fonctions décimales. Cette seule ligne suffisait à perdre de la précision. Le traitement utilise maintenant le même module que les autres calculs. Les nombres reçus des fournisseurs sont aussi conservés sous leur forme textuelle avant l'analyse du JSON.

Vérifier la réponse complète des fournisseurs

Certains fournisseurs renvoient un statut HTTP 200 avec un message de quota ou un prix nul. Les adaptateurs vérifient donc les champs et refusent un cours absent ou non strictement positif. L'échec déclenche le fournisseur suivant ou le dernier cours connu. Un prix à zéro ne doit pas être accepté comme une cotation valide.

Faire respecter les confirmations côté serveur

Une demande de mot de passe dans un dialogue ne protège pas la route si l'API accepte l'action sans ce mot de passe. La suppression de compte vérifie désormais l'empreinte côté serveur. Le contrôle d'interface guide l'utilisateur ; le contrôle serveur protège les données.

Vérifier l'installation et la configuration réelles

Un fichier de verrouillage incohérent a fait échouer l'installation stricte utilisée par Docker. Le corrigé a permis de retrouver une construction reproductible avec les dépendances retenues.

Lors d'un autre contrôle, les positions AAPL et NVDA n'avaient pas de cours parce que les clés n'étaient pas présentes dans l'environnement du conteneur exécuté. Le diagnostic initial orienté vers les quotas était incomplet. La configuration de ce conteneur a dû être vérifiée. Les captures récentes des positions montrent leur valorisation ; elles ne garantissent pas la disponibilité future des fournisseurs.

Ces difficultés m'ont appris à localiser plus vite l'origine d'un problème : interface, réponse HTTP, règle métier, base ou configuration. Une suite de tests réussie reste utile, mais ne remplace pas ces vérifications ciblées.

4.2 Bilan et perspectives

Ce qui est réalisé

WalletWatch réunit les sept écrans prévus : connexion, patrimoine, positions, détail d'une position, saisie d'un mouvement, seuils et compte. Le serveur prend en charge les calculs, le cloisonnement des données et les cours. La pile Docker et la procédure de déploiement sont fournies avec le code.

L'application répond au besoin de regrouper les actifs saisis et de calculer leur prix de revient. Les essais présentés dans le dossier permettent de comparer des résultats attendus et obtenus. Ils ne constituent pas une garantie d'absence de défaut dans toutes les situations.

Les limites du résultat

Le catalogue d'actions reste limité à une liste autorisée. Les cours dépendent de services tiers et de leurs quotas. Un dernier cours connu peut être ancien ; sa date et l'absence éventuelle de cours doivent rester visibles. L'application ne suit pas les liquidités d'un compte bancaire : le produit d'une vente ne devient pas un solde en euros dans le tableau de bord.

Les annonces et l'administration n'ont pas été intégrées aux parcours. Le schéma et le code conservent quelques éléments préparatoires, clairement distingués du périmètre livré. L'import de fichiers et l'export fiscal sont également hors du périmètre.

La pile présentée est une démonstration locale. Avant une ouverture à de vrais utilisateurs, il faudrait configurer HTTPS, prévoir l'envoi sécurisé des liens de récupération et vérifier les sauvegardes, les accès et l'exploitation du service. Le mode qui affiche une clé de récupération est réservé aux comptes de démonstration.

Ce que j'ai appris

Écrire les décisions m'a aidé à conserver un périmètre réalisable lorsque de nouvelles idées apparaissaient, notamment l'administration, les notifications et l'import de données.

J'ai aussi mieux distingué le rôle des tests et celui des vérifications en conditions réelles. Un test unitaire contrôle un calcul ou une branche du code ; une vraie base est nécessaire pour observer un verrou et deux écritures concurrentes, et le navigateur pour juger un formulaire ou une mise en page.

La centralisation des règles métier a été un autre apprentissage concret. Recalculer le prix de revient depuis les mouvements évite de maintenir plusieurs valeurs qui pourraient se contredire, mais impose de maîtriser l'ordre chronologique, les arrondis et le temps de calcul.

Les prochaines améliorations

L'import de mouvements serait utile pour éviter une longue ressaisie. Il faudrait présenter les lignes reconnues et les erreurs avant toute écriture. L'ouverture à des utilisateurs réels demanderait ensuite un travail sur l'hébergement, la récupération de compte et les contrôles d'exploitation.

J'aimerais aussi élargir les vérifications d'accessibilité et automatiser des parcours complets. Pour l'oral, je peux déjà m'appuyer sur trois cas que j'ai réellement travaillés : le verrou SQL, la clé de cache qui distingue le type d'actif et la correction des arrondis.

Annexes

Les extraits ont été confrontés au code du commit a8a5e6d. Les plages de lignes indiquent les coupes. Les maquettes et captures historiques représentent des étapes du développement et peuvent conserver le nom CapitAll. Les valeurs de démonstration sont fictives.

Annexe A : Maquettes des interfaces

Ces maquettes montrent la conception avant le changement de nom et les derniers ajustements. Les planches présentent l'ensemble ; les vues individuelles permettent de lire les principaux écrans.

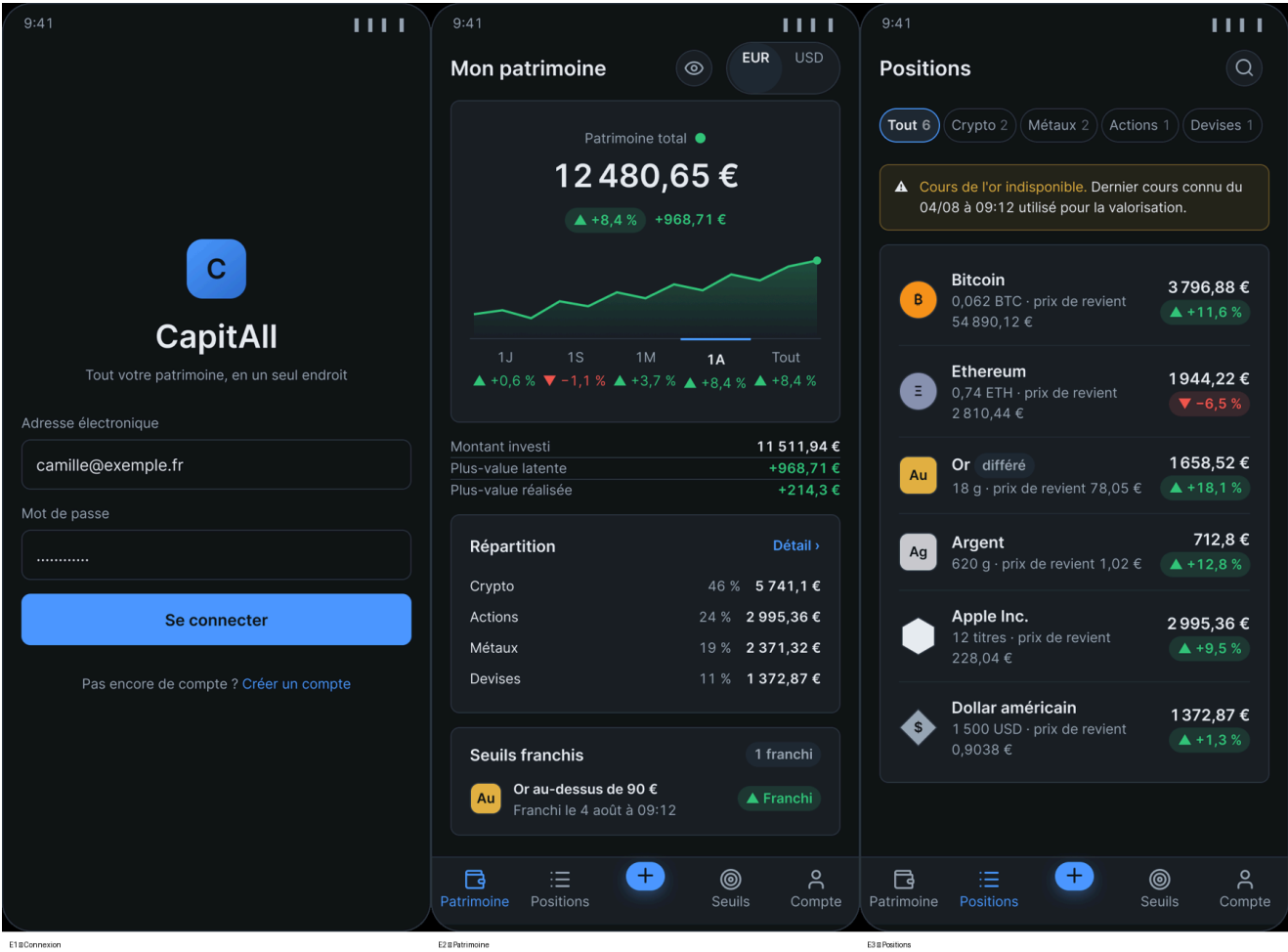


Figure 17 : Maquettes en 375 px : connexion, patrimoine, positions.

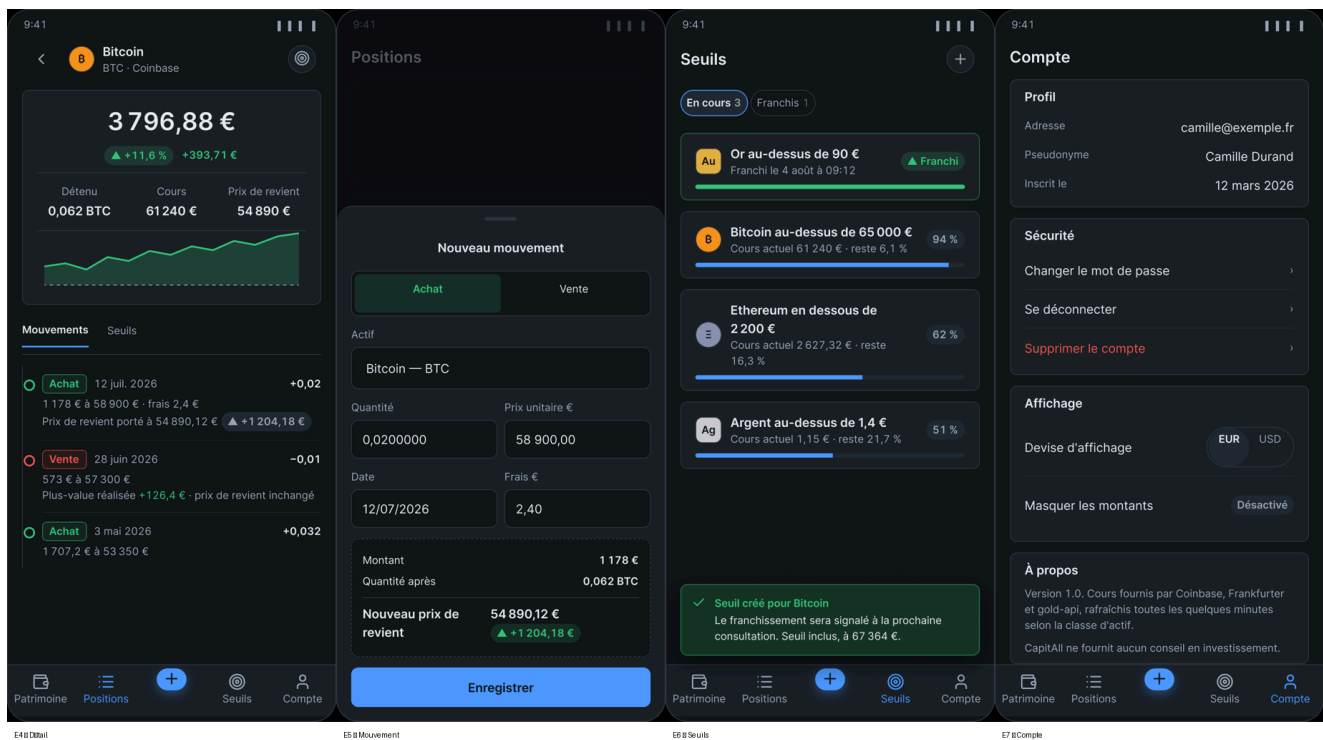
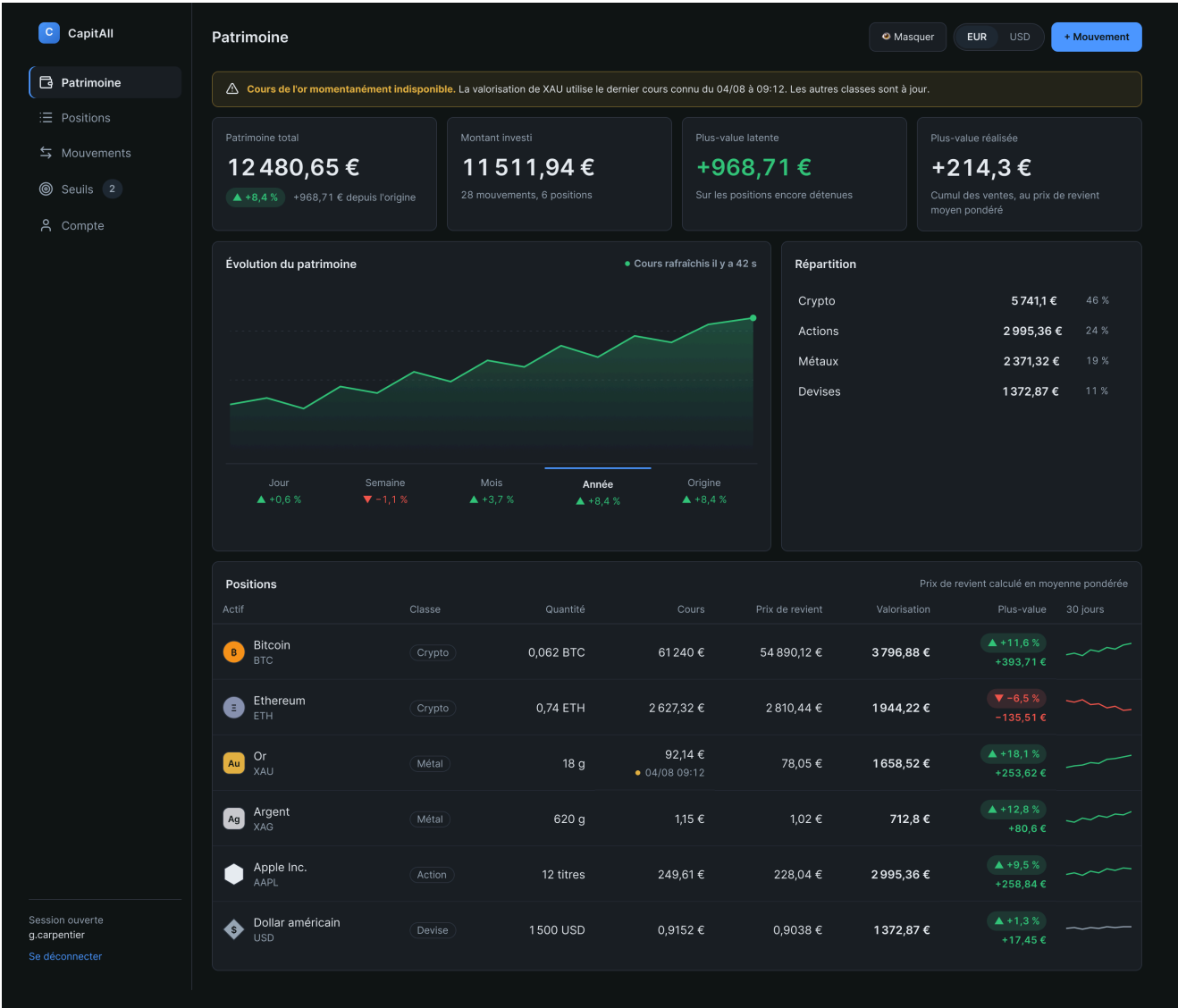


Figure 18 : Maquettes en 375 px : détail d'une position, mouvement, seuils, compte.



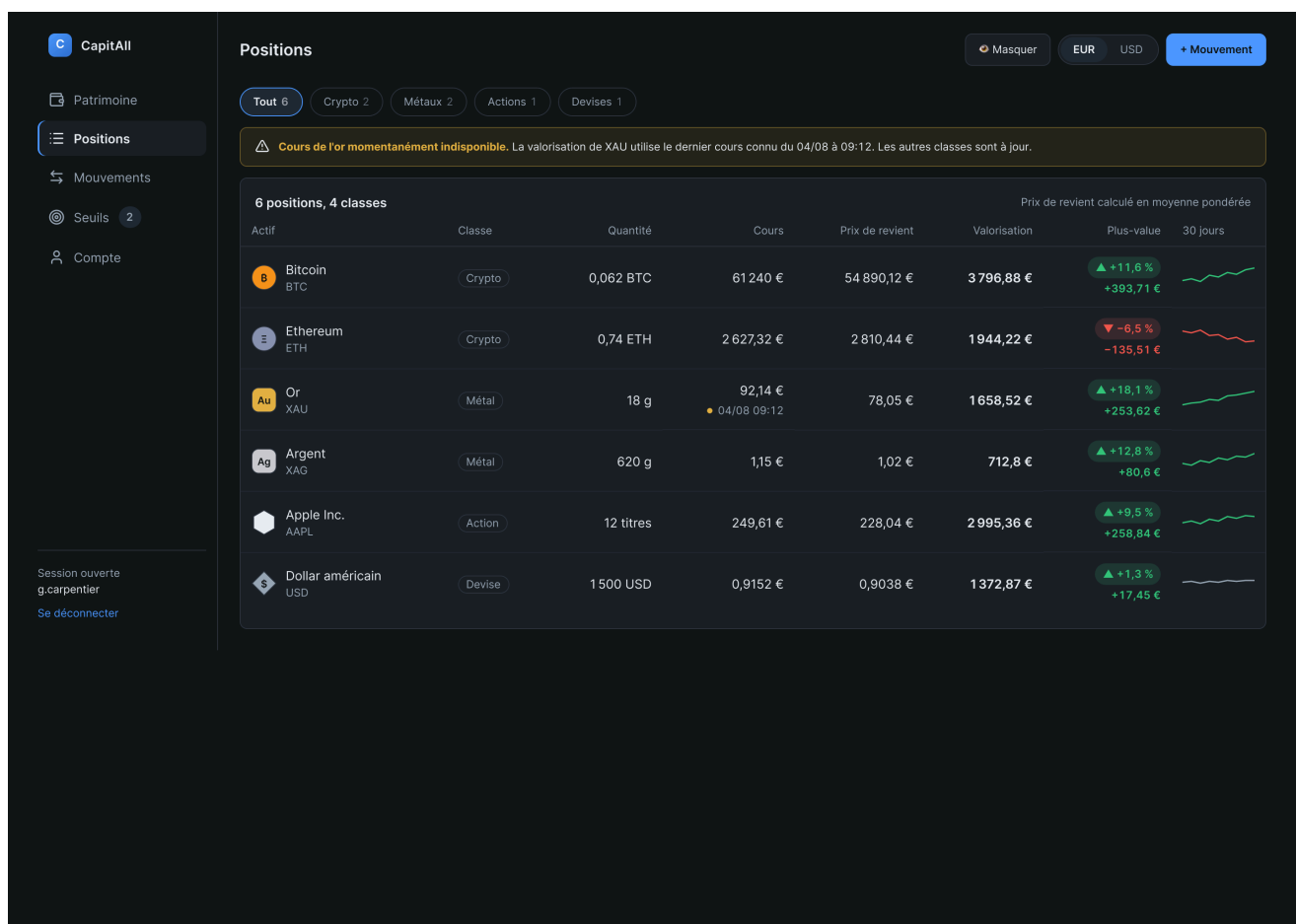


Figure 20 : Maquette de l'écran Positions en 1440 px.

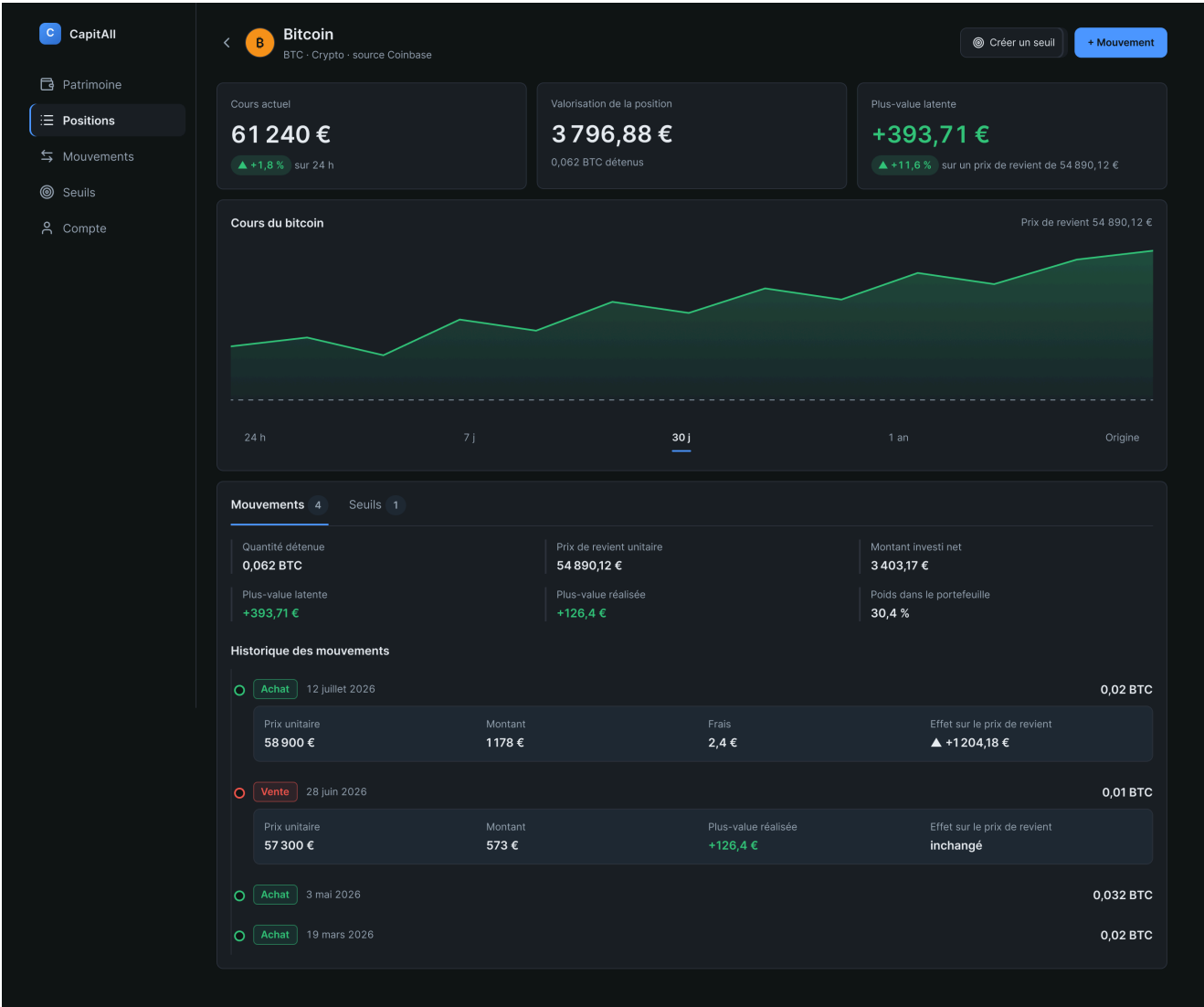


Figure 21 : Maquette du détail d'une position en 1440 px.

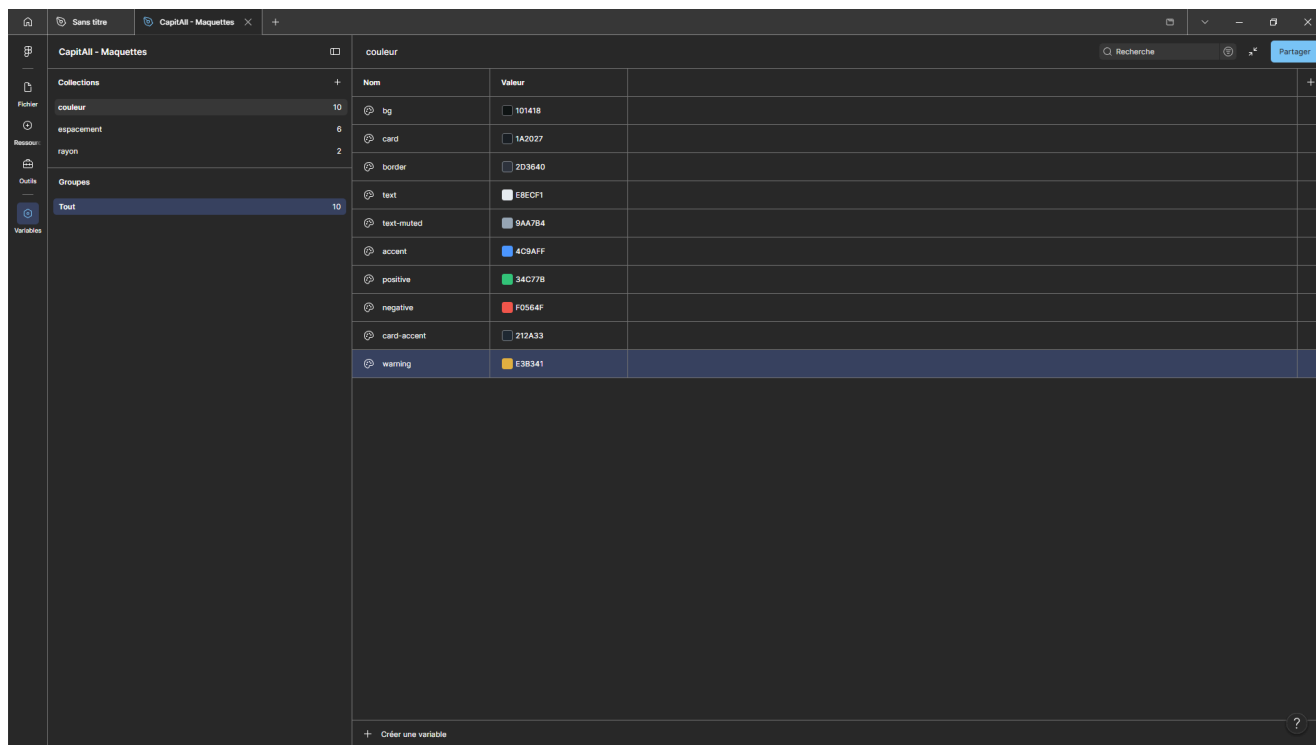


Figure 22 : Les collections de variables du fichier de maquettes : couleurs, espacements et rayons, reprises telles quelles en variables CSS.

Annexe B : Structure de la base

Le script complet se trouve dans `backend/db/schema.sql`. Plutôt que de le reproduire sur plusieurs pages, cette annexe résume le rôle des huit tables réellement créées et les contraintes principales.

Table	Rôle	Contraintes principales
utilisateur	compte et état d'accès	adresse unique, rôle limité, borne de révocation
actif	position suivie par un compte	propriétaire obligatoire, symbole unique par compte
transaction	achat, vente ou sortie	sens contrôlé, quantités et frais cohérents
alerte	seuil sur un actif ou le patrimoine	cible cohérente avec la présence d'un actif
snapshot_cours	cours quotidien d'un actif	une ligne par actif et par date
snapshot_valorisation	valeur quotidienne du patrimoine	une ligne par compte et par date
annonce	fonction préparée mais reportée	auteur obligatoire, publication datée
reinitialisation_mot_de_passe	clé temporaire de récupération	empreinte unique, expiration et utilisation datées

Les relations et les types exacts sont présentés par le schéma relationnel de la section 3.4. Les migrations datées complètent ce script lorsque la structure évolue.

Annexe C : Points d'entrée de l'API

La version présentée expose 25 couples méthode-route applicatifs et une route de santé. Les routes privées attendent `Authorization: Bearer <jeton>`. Les paramètres financiers sont transmis en chaînes décimales.

Routes publiques

Méthode	Route	Fonction
GET	/api/sante	Santé du service
POST	/api/auth/inscription	Création de compte
POST	/api/auth/connexion	Connexion
POST	/api/auth/mot-de-passe-oublie	Demande de récupération
POST	/api/auth/reinitialisation	Remplacement du mot de passe

Le projet n'envoie pas de courriel. L'affichage de la clé de récupération est une option de démonstration à ne pas activer pour des comptes réels.

Routes privées

Méthode	Route	Fonction
GET	/api/auth/moi	Profil courant
GET	/api/symboles	Catalogue
GET / POST	/api/actifs	Liste / création
GET / PATCH / DELETE	/api/actifs/:id	Détail / renommage / suppression
POST	/api/actifs/:id/transactions	Enregistrement
POST	/api/actifs/:id/transactions/simulation	Simulation
POST	/api/actifs/:id/transactions/:idTransaction/simulation	Simulation de correction
PATCH / DELETE	/api/actifs/:id/transactions/:idTransaction	Correction / suppression
GET	/api/portefeuille	Consultation
POST	/api/portefeuille/actualisation	Relevés et seuils
GET	/api/portefeuille/historique	Historique
GET / POST	/api/alertes	Liste / création de seuil
PATCH	/api/alertes/:id	Désactivation
PATCH	/api/compte/mot-de-passe	Changement et nouveau jeton (200)
GET	/api/compte/export-mouvements	CSV
DELETE	/api/compte	Suppression après vérification du mot de passe

alertes et transactions sont les noms techniques de l'API. L'interface emploie « seuils » et « mouvements ». Les contrats détaillés sont dans docs/api/.

Annexe D : Extraits techniques complémentaires

Le corps du dossier contient déjà les extraits nécessaires pour comprendre la fonctionnalité représentative. Cette annexe conserve seulement deux compléments courts : la règle d'arrondi décimal et l'exécution d'une transaction SQL. Les fichiers complets restent disponibles dans le dépôt au commit de référence.

D.1 : Arrondi d'une division entière

backend/src/Utils/decimal.js, lignes 151 à 172.

```
function diviserEntiers(numerateur, denominateur) {
  if (denominateur === 0n) {
    throw new Error('Division par zéro dans un calcul décimal.');
```

```
  }

  const quotient = numerateur / denominateur;
  const reste = numerateur % denominateur;

  if (reste === 0n) {
    return quotient;
  }

  const resteDouble = (reste < 0n ? -reste : reste) * 2n;
  const denominateurAbsolu = denominateur < 0n ? -denominateur : denominateur;

  if (resteDouble < denominateurAbsolu) {
    return quotient;
  }

  const memeSigne = numerateur < 0n === denominateur < 0n;
  return memeSigne ? quotient + 1n : quotient - 1n;
}
```

D.2 : Exécution d'une transaction SQL

backend/src/db/index.js, lignes 30 à 45.

```
async function executerDansTransaction(operation) {
  const client = await pool.connect();
  const queryTransactionnelle = client.query.bind(client);

  try {
    await client.query('BEGIN');
    const resultat = await operation(queryTransactionnelle);
    await client.query('COMMIT');
    return resultat;
  } catch (erreur) {
    await client.query('ROLLBACK');
    throw erreur;
  } finally {
    client.release();
  }
}
```

Annexe E : Captures complémentaires

Les vues suivantes ont été prises le 10 septembre 2026 sur l'instance locale, avec des données fictives. Les formulaires de correction ont été ouverts sans valider de changement. Les vues mobiles correspondent à une largeur de 375 pixels et peuvent nécessiter un défilement dans l'application.

Trois défauts d'affichage restent visibles sur ces captures et n'ont pas été corrigés dans le code du commit de référence. Le formulaire de correction montre les zéros de la précision SQL, par exemple une quantité écrite 1.9980000000000000. Le formulaire d'une sortie conserve l'aide rédigée pour les frais d'achat, qui ne s'applique pas à cette opération. Enfin, le rappel du cours du jour est précédé d'un point isolé, conséquence d'un assemblage de deux phrases ; il se lit aussi sur les figures 10 et 15. La correction demanderait de retirer à l'affichage les seuls zéros inutiles en conservant les valeurs sous forme de chaînes décimales, d'adapter le texte d'aide au sens de l'opération et de reprendre cet assemblage. Je les signale plutôt que de retoucher les images.

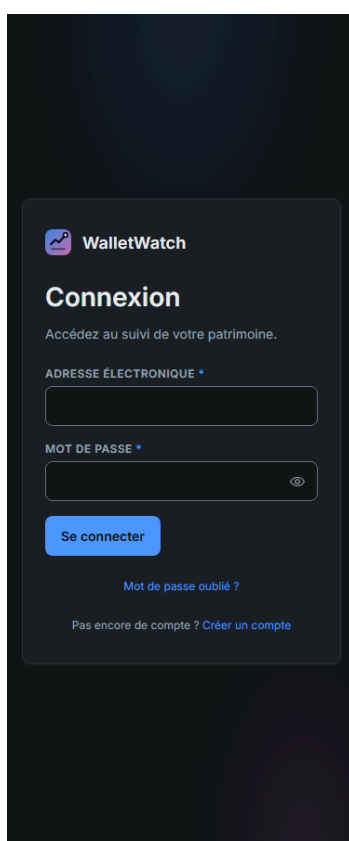
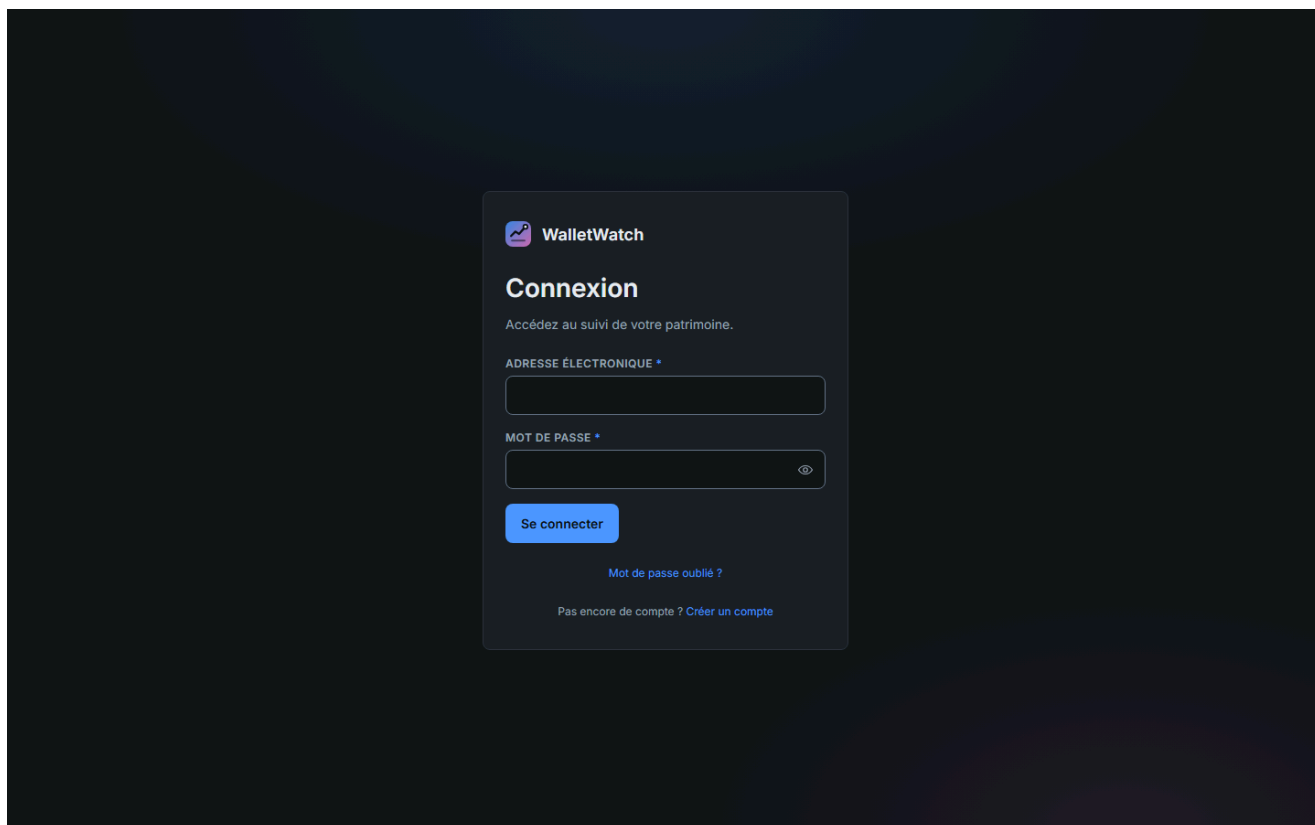


Figure 23 : Connexion sur bureau puis sur téléphone. Les champs et les actions gardent le même ordre.

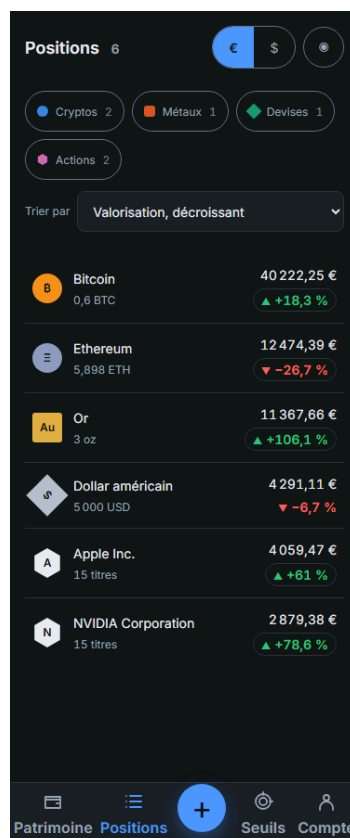
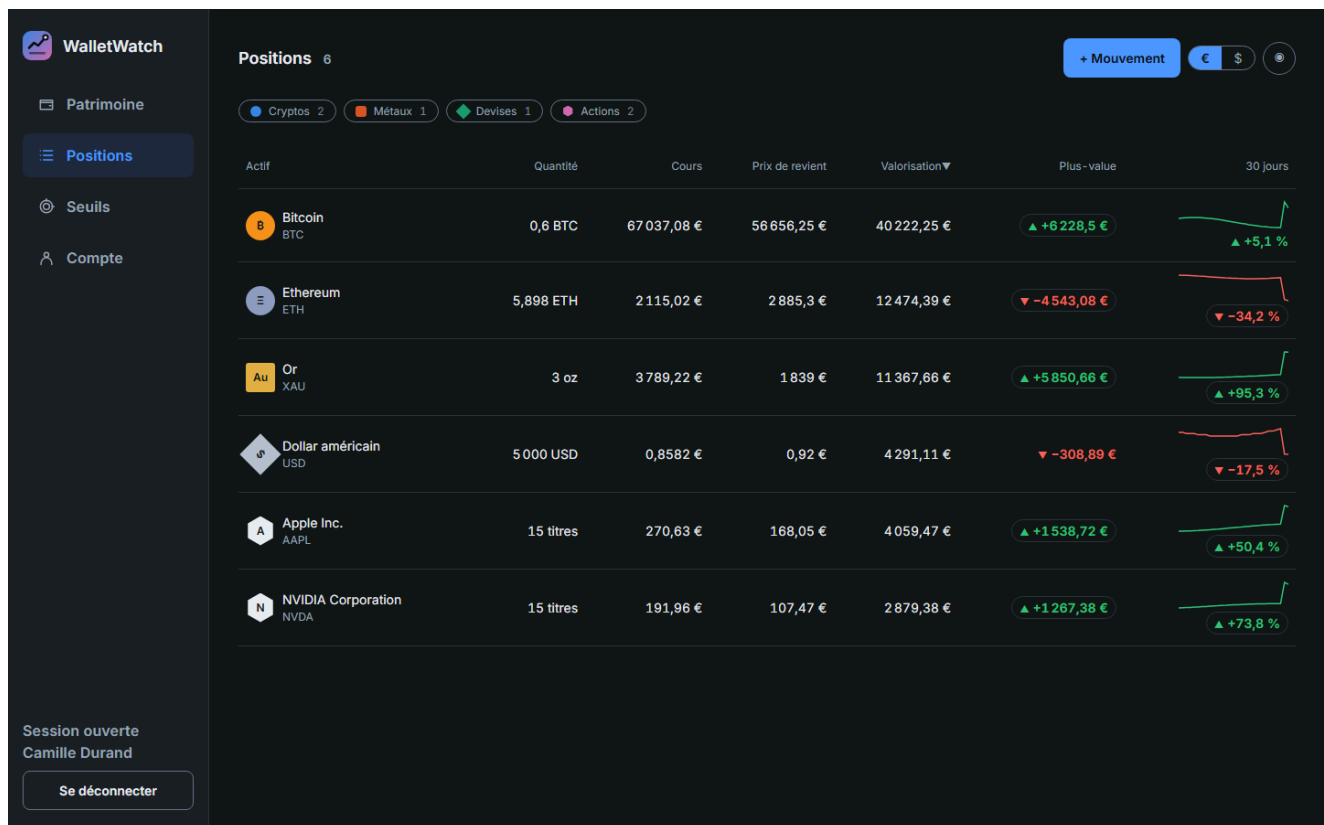


Figure 24 : Positions : tableau sur bureau et cartes sur téléphone. Les six positions du jeu de démonstration comprennent Apple et NVIDIA.

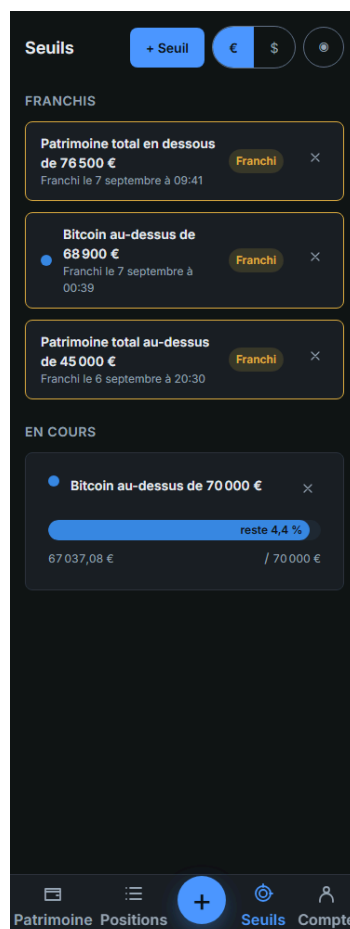
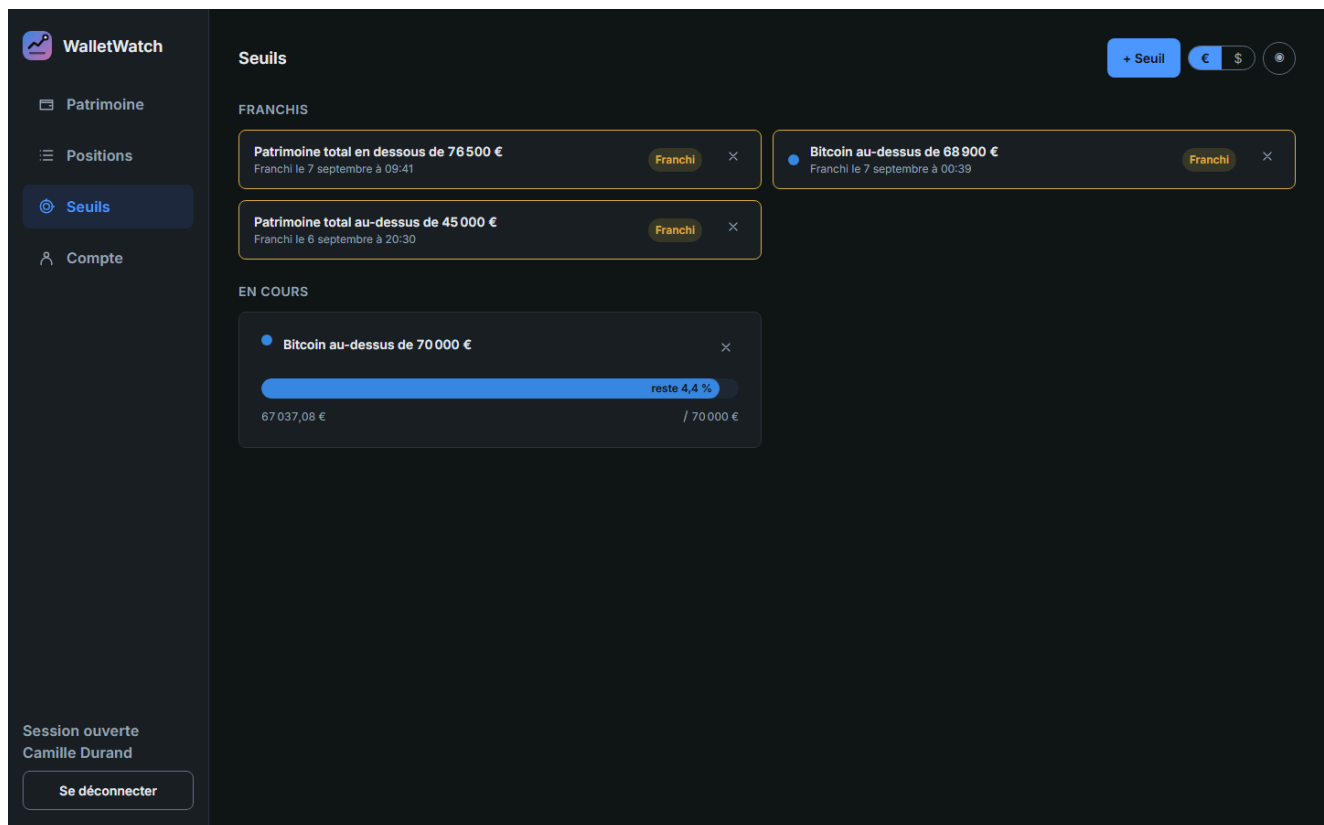


Figure 25 : Seuils sur bureau puis sur téléphone : distinction entre seuils franchis et seuil en cours. Un franchissement passé reste visible.

 WalletWatch

Patrimoine

Positions

Seuils

Compte

Session ouverte
Camille Durand

Se déconnecter

Compte

Informations

Adresse électronique

user@capitali.fr

Pseudonyme

Camille Durand

Inscrit depuis le

06 septembre 2026

Rôle

Utilisateur

Ces informations ne sont pas modifiables.

Sécurité

MOT DE PASSE ACTUEL *

NOUVEAU MOT DE PASSE *

Dix caractères au minimum.

CONFIRMER LE NOUVEAU MOT DE PASSE *

Changer le mot de passe

Se déconnecter

Supprimer mon compte

Figure 26 : Compte : profil et formulaire de changement de mot de passe. Les préférences et l'export se trouvent plus bas, hors de cette capture.

Bitcoin

Renommer

Nouveau mouvement

SENS DE L'OPÉRATION

Achat Vente

Sortie

ACTIF *

Bitcoin — BTC (Cryptos)

B Cryptos [Suivre un nouvel actif](#)

QUANTITÉ *

0,2

PRIX UNITAIRE (€) *

58900

① Cours du jour : 67 042,62 €
 . Conservez-le pour une opération du moment,
 corrigez-le pour une opération passée.

DATE DE L'OPÉRATION *

10/09/2026

FRAIS (€)

2,40

Facultatif. Les frais d'achat entrent dans le prix de revient.

CES FRAIS ONT ÉTÉ PRÉLEVÉS EN

euros

EFFET DE CE MOUVEMENT

Montant de l'opération	11 780 €
Frais	2,4 €

Figure 27 : Saisie du même achat de Bitcoin sur téléphone. Le bas du récapitulatif et les boutons se consultent par défilement dans la feuille.

Patrimoine

Positions

Seuils

Compte

Session ouverte

Camille Durand

Se déconnecter

Ethereum

ETH - Cryptos

Valorisation de la position

12 474,39 €

GAINS LATENTS

-4 543,08 €

Jour

3212€

3015€

2715€

2415€

2115€

2026-08-11

2026-08-11

Mouvements

3

Seuils

Sortie

25 août 2026

Valeur sortie du portefeuille

288,53 €

Corriger

Supprimer

Achat

2 août 2026

Prix unitaire

3 150 €

Corriger

Supprimer

Achat

18 juin 2026

Prix unitaire

2 750 €

11 000 €

6 €

Corriger le mouvement

×

SENS DE L'OPÉRATION

Achat

Vente

Sortie

ACTIF

Ethereum — ETH (Cryptos)

Cryptos

QUANTITÉ

1.998000000000000000

PRIX UNITAIRE (€)

3150.000000000000000000

① Cours du jour : 2 115,02 €

Conservez-le pour une opération du moment, corrigez-le pour une opération passée.

DATE DE L'OPÉRATION

02/08/2026

FRAIS PRÉLEVÉS

0.002000000000000000

Facultatif. Les frais d'achat entrent dans le prix de revient.

CES FRAIS ONT ÉTÉ PRÉLEVÉS EN

ETH, l'actif de l'opération

Saisissez la quantité reçue et les frais retenus par la plateforme : ils sont convertis au prix de l'opération.

EFFET DE CE MOUVEMENT

Montant de l'opération

6 293,7 €

Frais

6,3 € 0,002 ETH

Quantité détenue après

5,898 ETH

Nouveau prix de revient

2 885,3 € Inchangé

Figure 28 : Correction d'un achat existant : les 0,002 ETH de frais et leur contre-valeur sont conservés. Formulaire consulté sans enregistrer de correction.

Patrimoine

Positions

Seuils

Compte

Session ouverte

Camille Durand

Se déconnecter

Ethereum

ETH - Cryptos

Valorisation de la position

12 474,39 €

GAINS LATENTS

-4 543,08 €

Jour

3212€

3015€

2715€

2415€

2115€

Prix de revient 2 885,3 €

2026-08-11

2026-08-11

Mouvements 3

Sortie

25 août 2026

Valeur sortie du portefeuille

288,53 €

Corriger

Supprimer

Achat

2 août 2026

Prix unitaire

3 150 €

Corriger

Supprimer

Achat

18 juin 2026

Prix unitaire

2 750 €

Montant

11 000 €

Frais

6 €

Effet sur le prix de revient

+2 751,5 €

5,898 ETH

2 115,02 €

2 885,3 €

Année

-27,1 %

Suivi

-27,1 %

-0,1 ETH

+1,998 ETH

Effet sur le prix de revient

+133,8 €

+4 ETH

Effet sur le prix de revient

+2 751,5 €

Corriger le mouvement

SENS DE L'OPÉRATION

Achat

Vente

Sortie

ACTIF *

Ethereum — ETH (Cryptos)

Cryptos

QUANTITÉ *

0.100000000000000000

Tout sortir

Une sortie retire la quantité de votre position sans contrepartie en euros : transfert vers un autre portefeuille, retrait, ou frais réglés en nature. Elle ne dégage aucune plus-value, et la valeur qui quitte le portefeuille est comptée à part.

DATE DE L'OPÉRATION *

25/08/2026

FRAIS (€)

Facultatif. Les frais d'achat entrent dans le prix de revient.

EFFET DE CE MOUVEMENT

Valeur sortie du portefeuille

288,53 €

Quantité détenue après

5,898 ETH

Nouveau prix de revient

2 885,3 € inchangé

Annuler

Enregistrer la correction

Figure 29 : Formulaire de correction ouvert sur une sortie de 0,1 ETH : elle réduit la quantité sans produire de plus-value réalisée, et la valeur sortie est présentée séparément. Aucune modification n'a été enregistrée.

60 / 61



Figure 30 : Entrée du parcours de récupération sur téléphone. Le projet ne fournit pas encore un canal d'envoi adapté à de vrais comptes ; le mode affichant une clé reste réservé à la démonstration.

Annexe F : Scénarios du jeu d'essai

Le fichier backend/src/services/__tests__/calculPortefeuille.test.js contient l'ensemble des tests exécutables. Le tableau reprend les scénarios les plus faciles à vérifier à la main, sans recopier plusieurs pages de code.

Scénario	Données principales	Résultat attendu
Position vide	aucun mouvement	quantité 0, prix de revient 0, plus-value 0 €
Frais d'achat	10 titres à 100 € et 5 € de frais	prix de revient 100,50 €, coût total 1 005 €
Moyenne pondérée	10 titres à 100 € puis 10 à 120 €, 5 € de frais à chaque achat	quantité 20, prix de revient 110,50 €, coût 2 210 €
Vente partielle	vente de 5 titres à 150 € avec 5 € de frais	quantité 15, prix de revient inchangé, plus-value réalisée 192,50 €
Vente à perte	vente de 5 titres à 80 € après un achat à 100 €	plus-value réalisée -100 €
Vente totale puis rachat	vente des 10 titres, puis achat de 4 titres à 200 €	nouvelle position de 4 titres avec un prix de revient de 200 €
Saisie dans le désordre	mêmes mouvements fournis dans un autre ordre	résultat identique après tri chronologique

Les résultats datés de l'ensemble des suites restent présentés dans la section 3.9.